

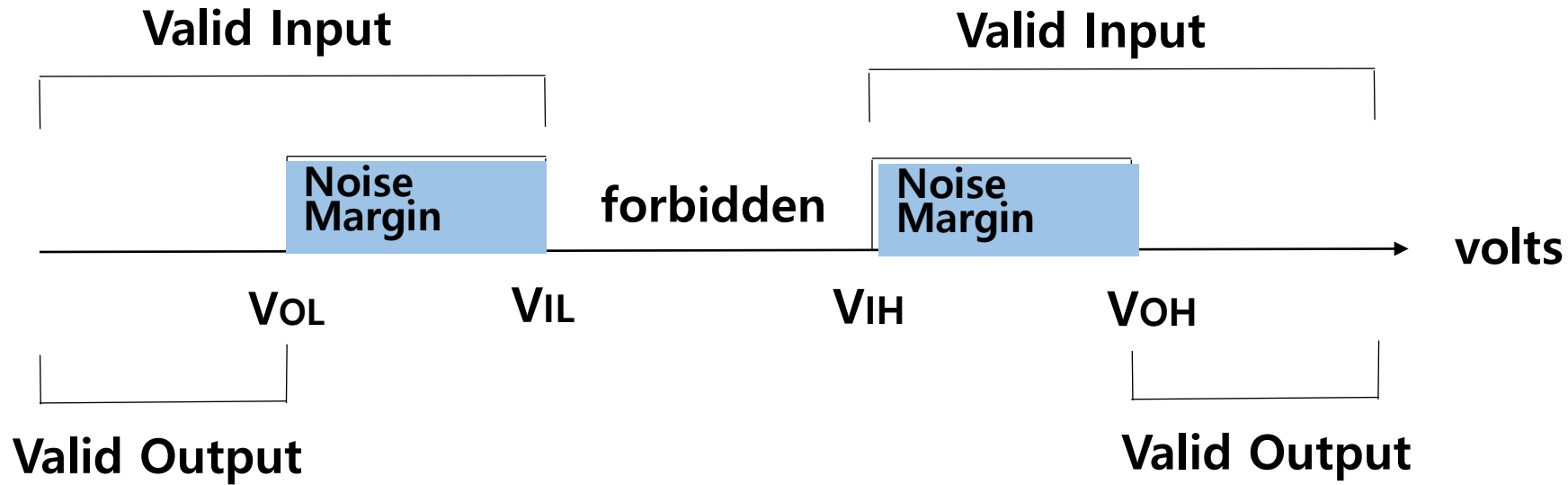
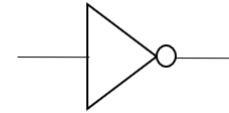
Sequential logic Finite State Machine

Computer Architecture

Last Lecture Recap

Digital Processing Element: Amplification

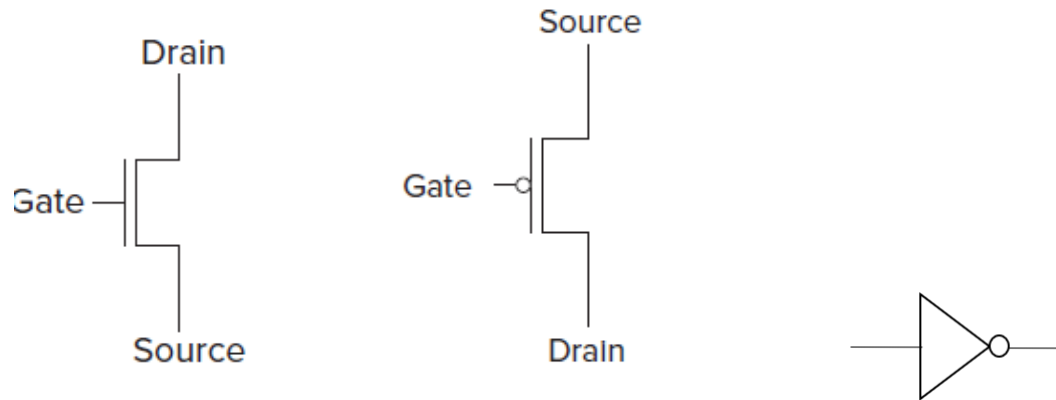
- Noise can change voltages slightly



- Silicon has wide noise margins

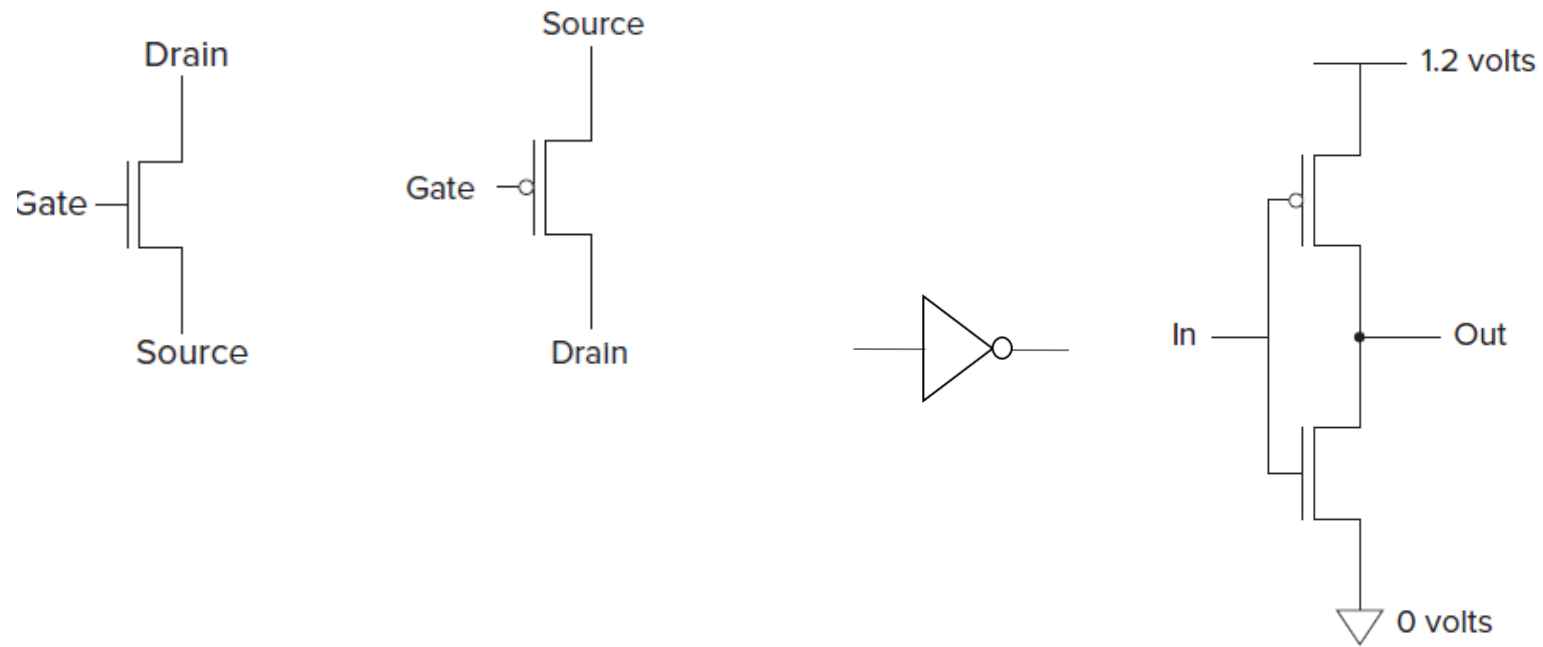
Transistors

- Logic gates are composed of transistors



Transistors

- Logic gates are composed of transistors



Logic simplification

- OR rules
- AND rules
- Commutative
- Associative
- Distributive
- Complements
- Absorption
- Reduction
- DeMorgan's Law

$$a+1=1, \quad a+0=a, \quad a+a=a$$

$$a1=a, \quad a0=0, \quad aa=a$$

$$a+b=b+a, \quad ab=ba$$

$$(a+b)+c=a+(b+c), \quad (ab)c=a(bc)$$

$$a(b+c)=ab+ac, \quad a+bc=(a+b)(a+c)$$

$$a+\bar{a}=1, \quad a\bar{a}=0$$

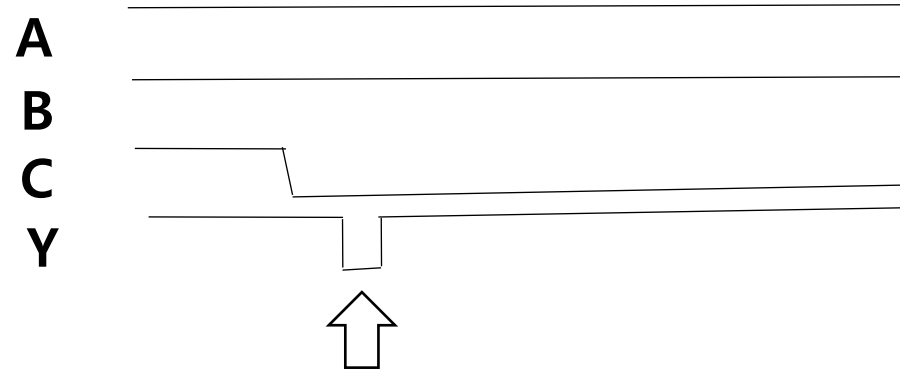
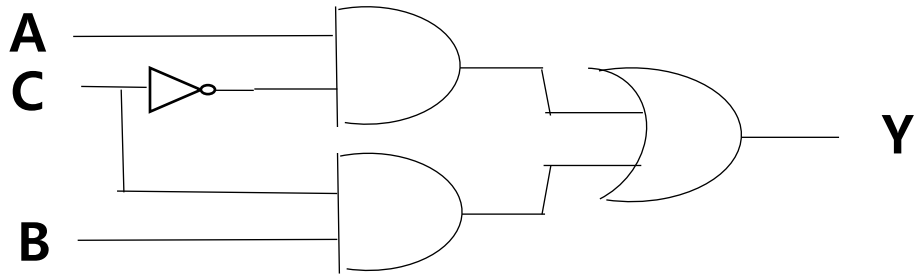
$$a+ab=a, \quad a+\bar{a}b=a+b, \quad a(a+b)=a, \quad a(\bar{a}+b)=ab$$

$$ab+\bar{a}b=b, \quad (a+b)(\bar{a}+b)=b$$

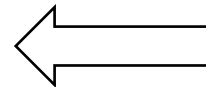
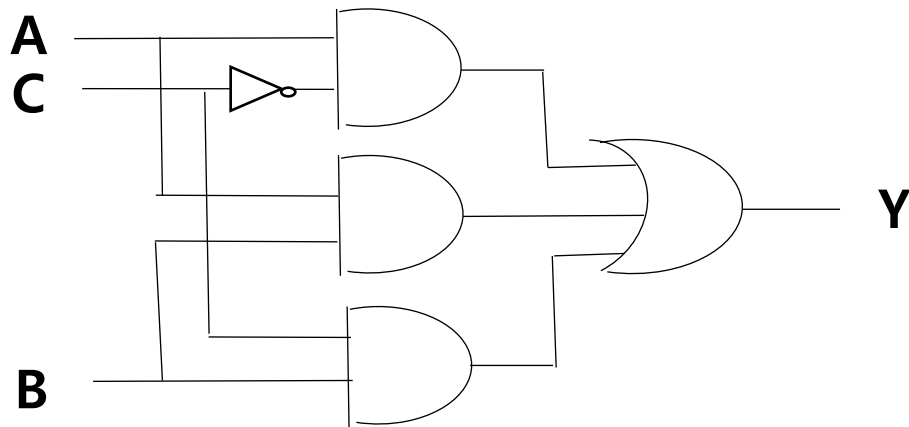
$$\overline{ab}=\bar{a}+\bar{b}, \quad \overline{a+b}=\bar{a}\bar{b}$$

Case for non-minimal SOP

- $Y = \bar{C}A + CB$



- $Y = \bar{C}A + AB + CB$



Lenient

Finding prime implicants on K-map

- Prime implicants
 - Rectangular region of the K-map with value 1
 - Start from $2^n, 2^{n-1}, \dots, 2$
 - Can overlap other implicants
 - A prime implicant is not completely contained in any implicants

C:BA	00	01	11	10
0	1	1	1	1
1	1	1	1	1

C:BA	00	01	11	10
0	0	1	1	0
1	1	1	1	1

C:BA	00	01	11	10
0	1	0	1	1
1	1	0	1	1

C:BA	00	01	11	10
0	0	0	1	1
1	1	0	1	1

What if?

- 4 Input Karnaugh map

What if?

- 4 Input Karnaugh map

DC BA	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	0	0	1

What if?

- 4 Input Karnaugh map

DC BA	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	0	0	1

- Minimal SOP

What if?

- 4 Input Karnaugh map

DC BA	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	0	0	1

- Minimal SOP

$$Y = C + A\bar{D} + \bar{A}D + B\bar{D}$$

What if?

- 4 Input Karnaugh map

DC BA	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	0	0	1

- Minimal SOP

$$Y = C + A\overline{D} + \overline{A}D + B\overline{D}$$

$$Y = C + A\overline{D} + \overline{A}D + \overline{A}B$$

What if?

- 4 Input Karnaugh map

DC BA	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	0	0	1

- Minimal SOP

$$Y = C + A\overline{D} + \overline{A}D + \overline{B}D$$

$$Y = C + A\overline{D} + \overline{A}D + \overline{A}B$$

- Lenient SOP

What if?

- 4 Input Karnaugh map

DC BA	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	0	0	1

- Minimal SOP

$$Y = C + A\overline{D} + \overline{A}D + \overline{B}D$$

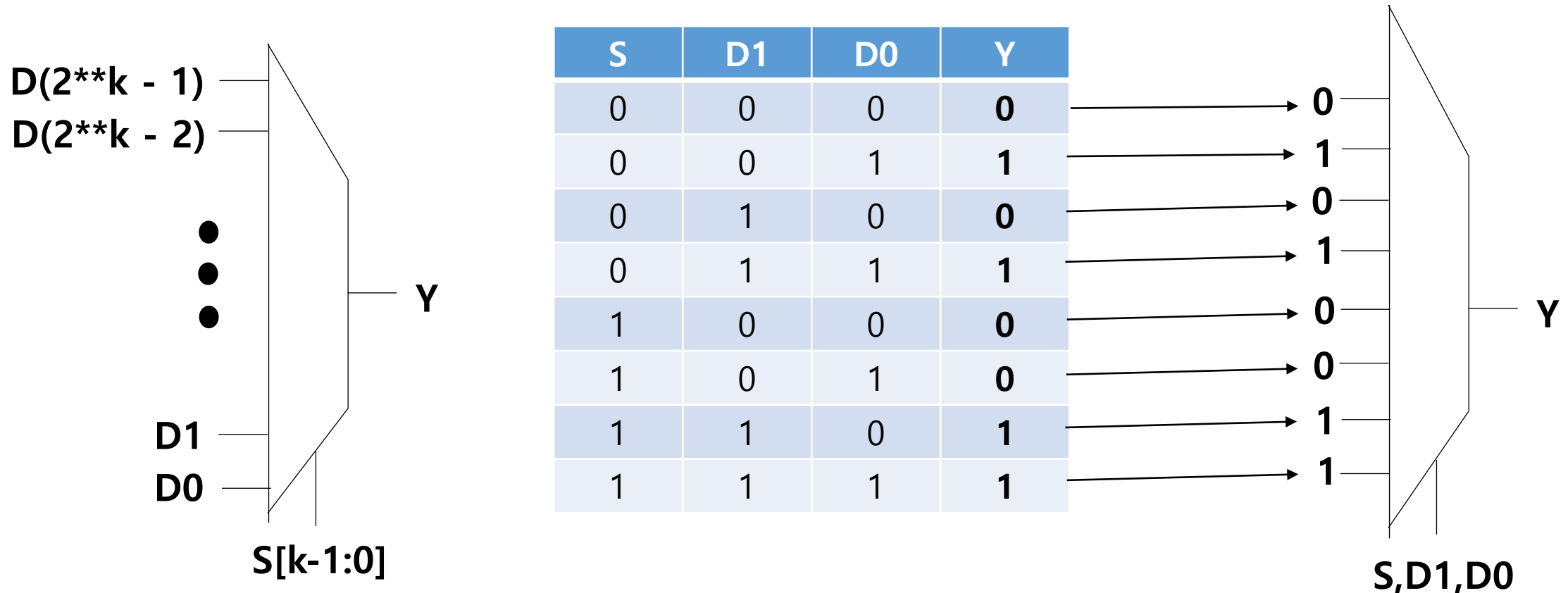
$$Y = C + A\overline{D} + \overline{A}D + \overline{A}B$$

- Lenient SOP

$$Y = C + A\overline{D} + \overline{A}D + \overline{B}D + \overline{A}B$$

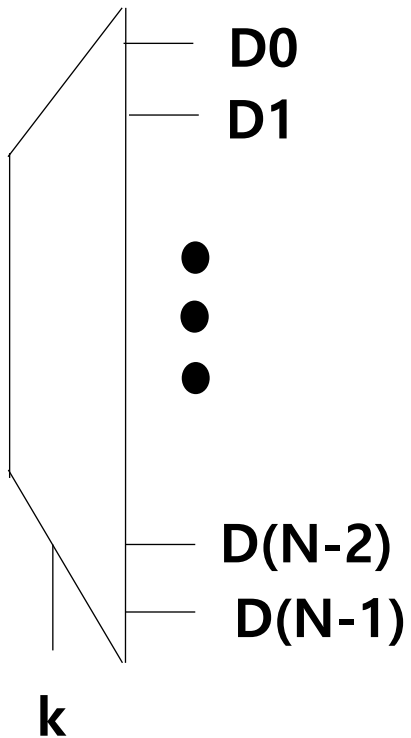
Multiplexers

- MUXes can be generalized to 2^k data inputs and k select inputs
 - Implementing some arbitrary Boolean functions using ONLY MUXes
 - Good for prototyping but not fast
 - Not practical if k is big



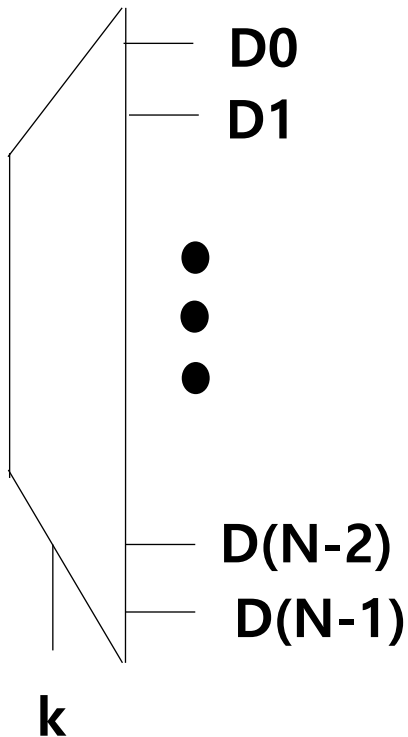
Decoders and ROM

- **Decoder**
 - k select inputs
 - $N=2^k$ outputs
 - One of D_s is HIGH
- **Read Only Memory (ROM)**

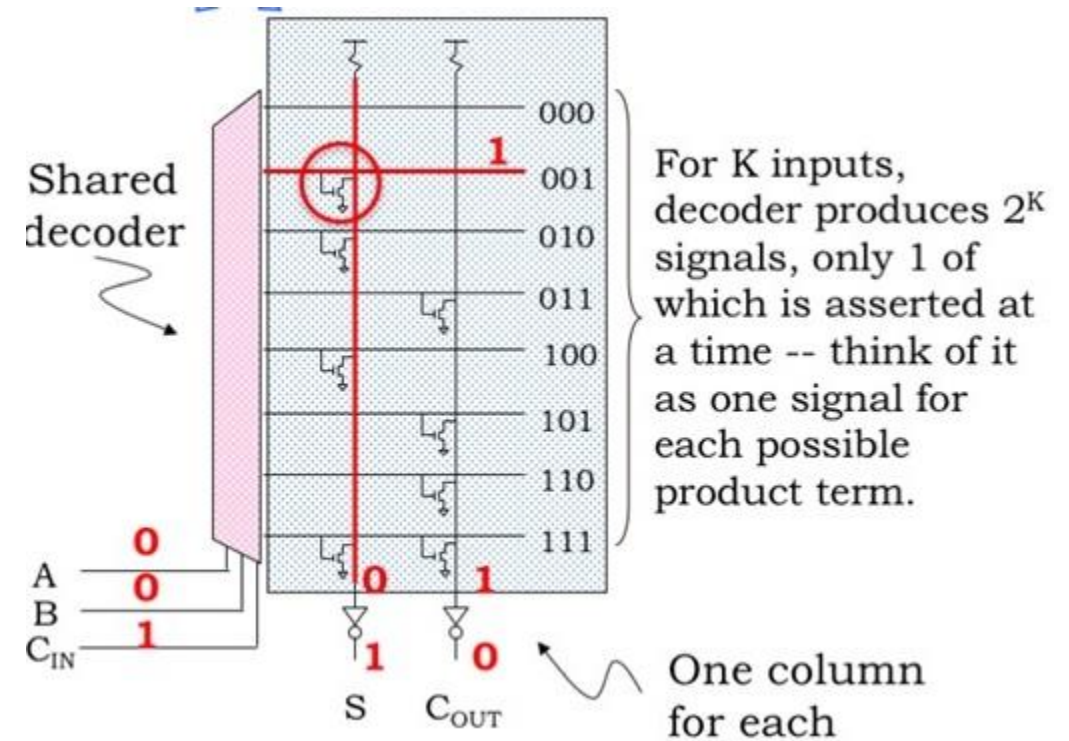
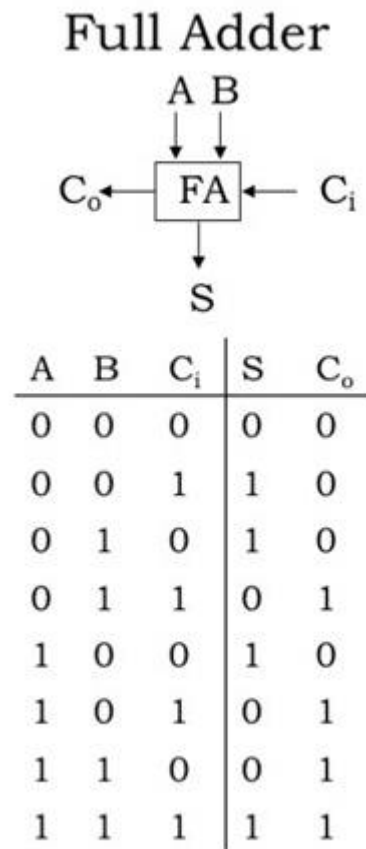


Decoders and ROM

- Decoder
 - k select inputs
 - $N=2^k$ outputs
 - One of D_s is HIGH



- Read Only Memory (ROM)



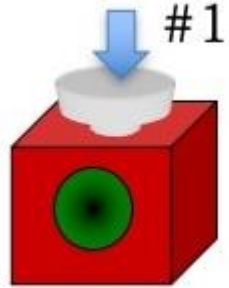
Sequential logic

Computer Architecture

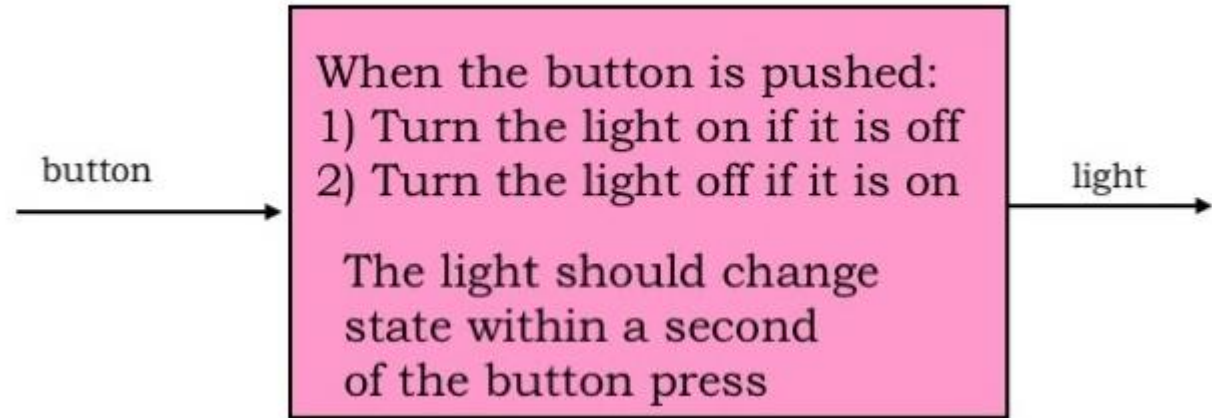
Readings

- *David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)*
 - Chapter 3

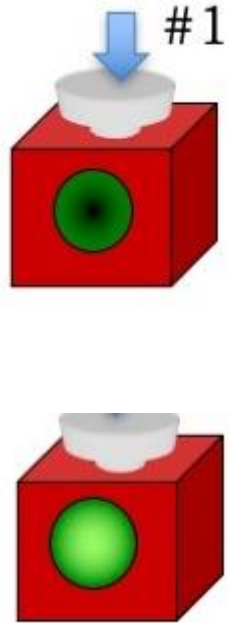
Digital State



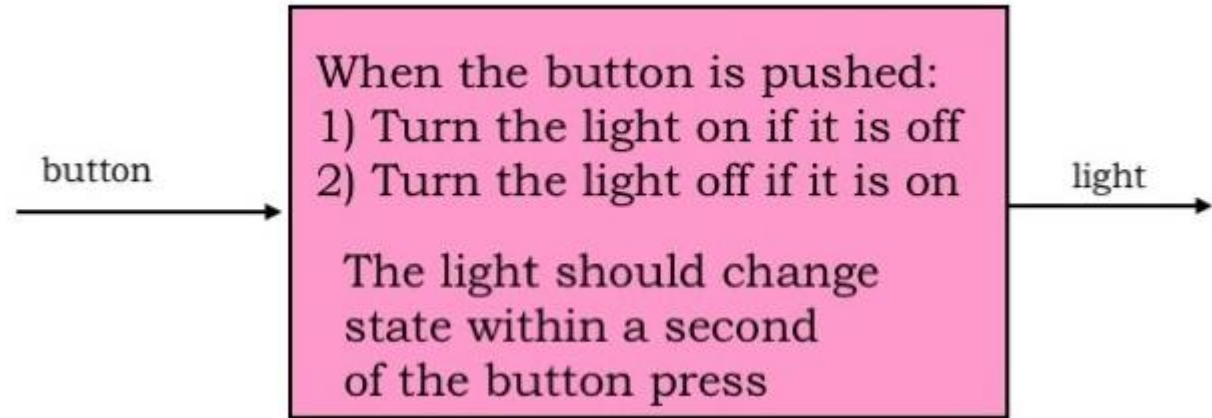
What if you were given the following design specification:



Digital State

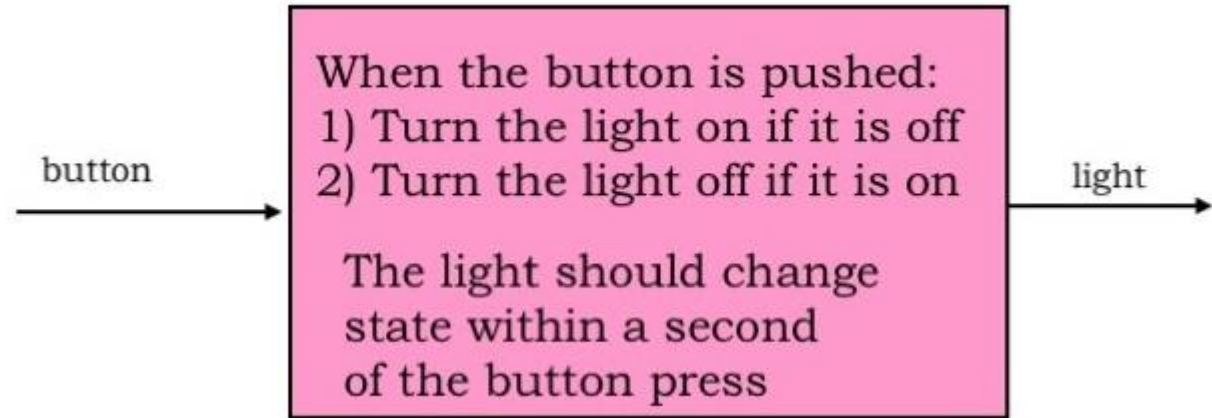
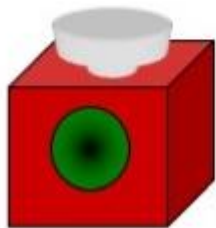
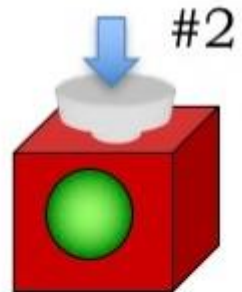
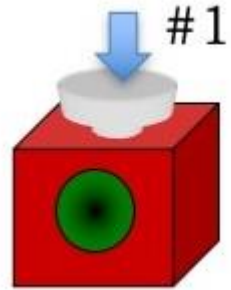


What if you were given the following design specification:

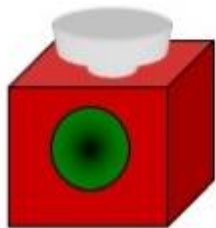
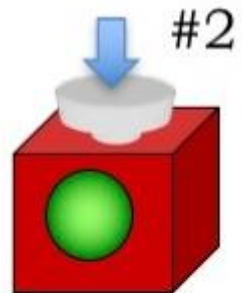
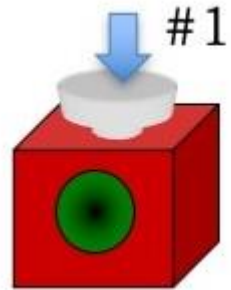


Digital State

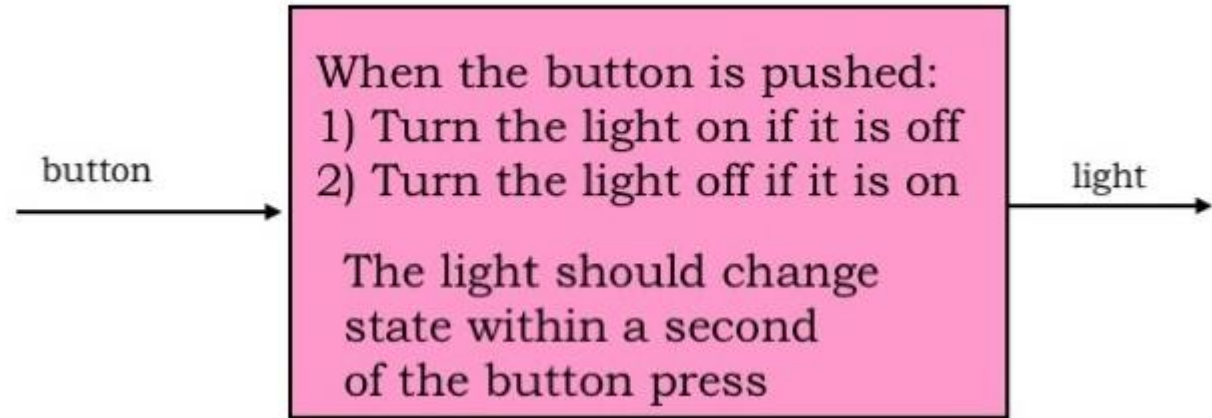
What if you were given the following design specification:



Digital State

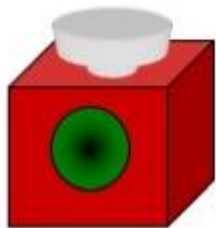
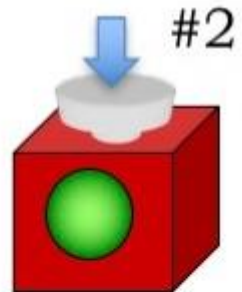
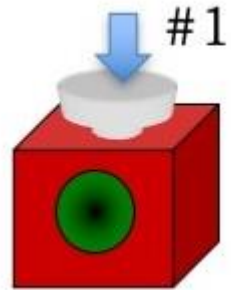


What if you were given the following design specification:

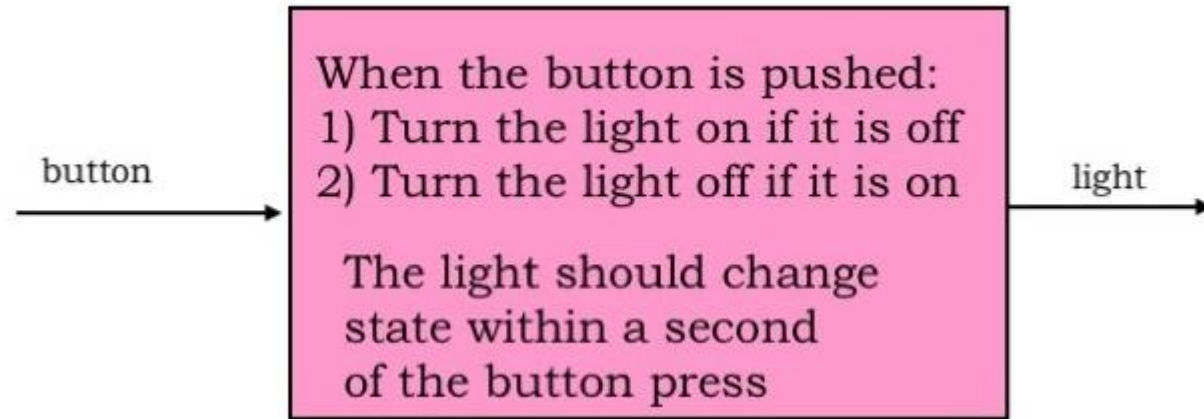


What makes this device different from those we've discussed before?

Digital State



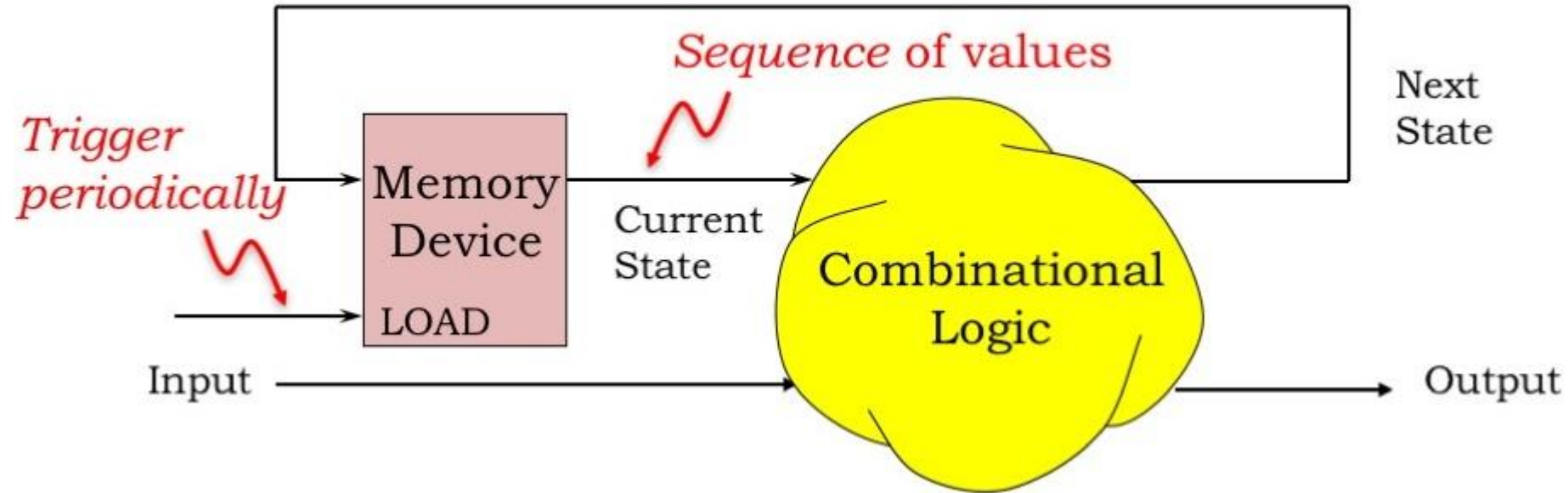
What if you were given the following design specification:



What makes this device different from those we've discussed before?

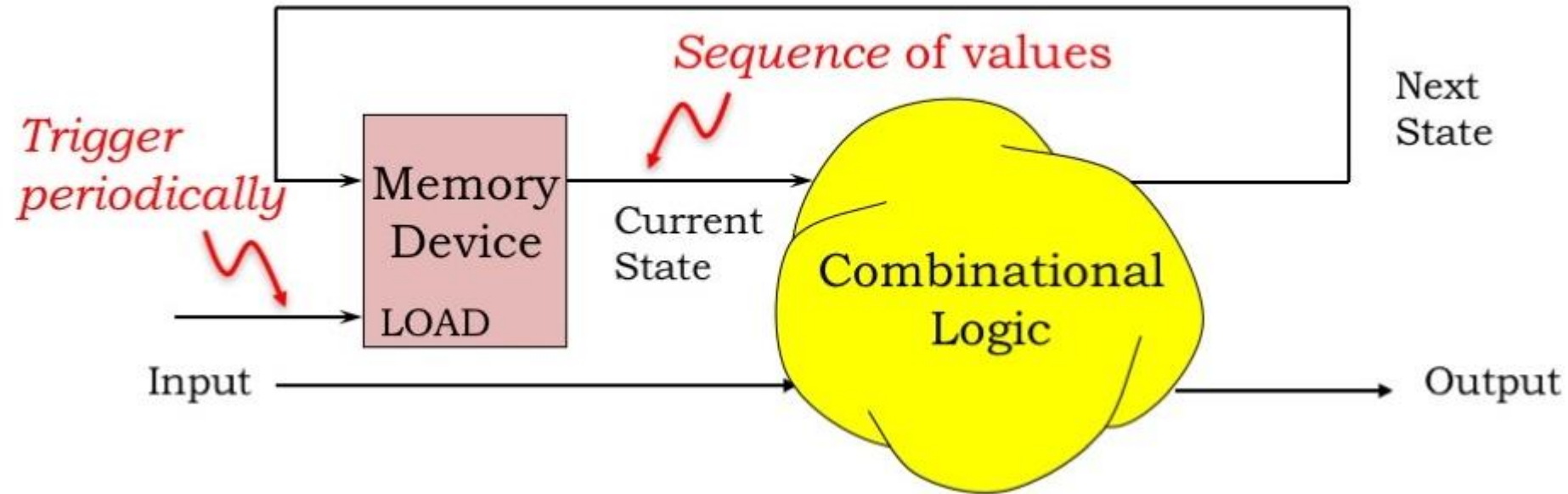
“State” – i.e., the device has memory

Digital State



Plan: Build a Sequential Circuit with stored digital STATE –

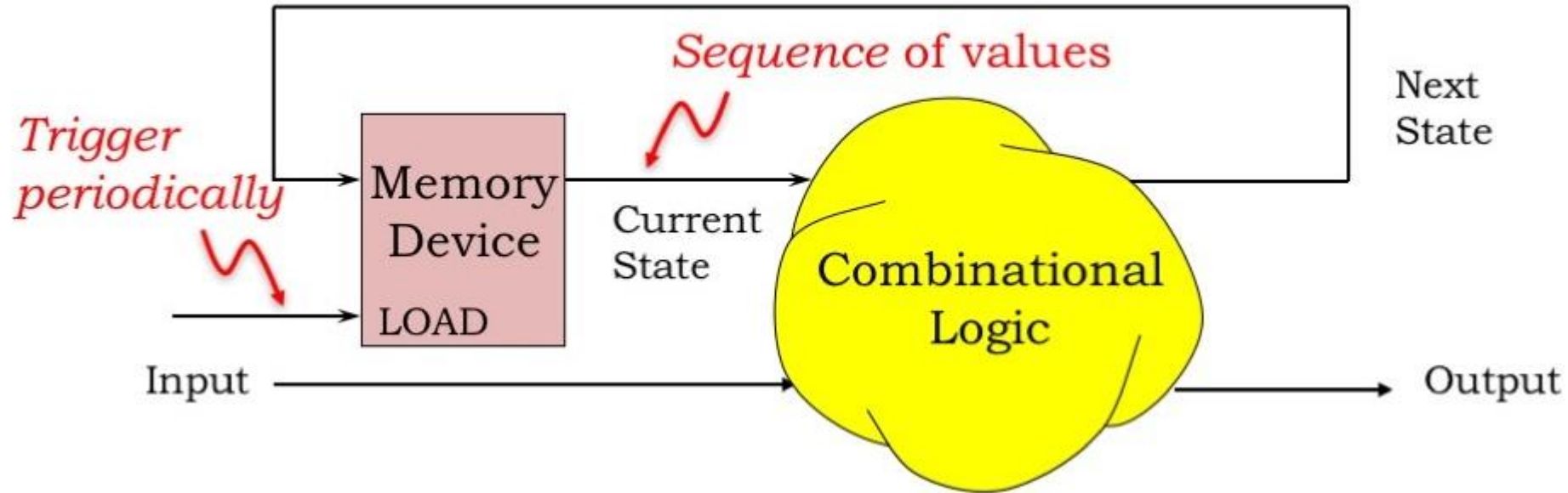
Digital State



Plan: Build a Sequential Circuit with stored digital STATE –

- Memory stores CURRENT state, produced at output

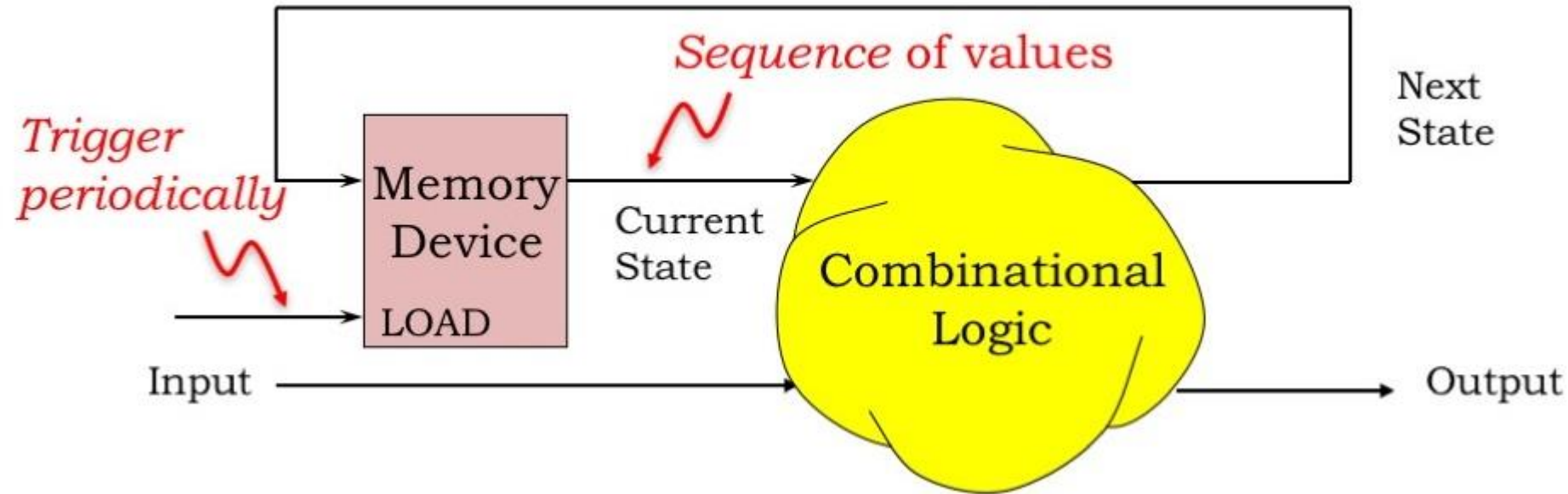
Digital State



Plan: Build a Sequential Circuit with stored digital STATE –

- Memory stores CURRENT state, produced at output
- Combinational Logic computes

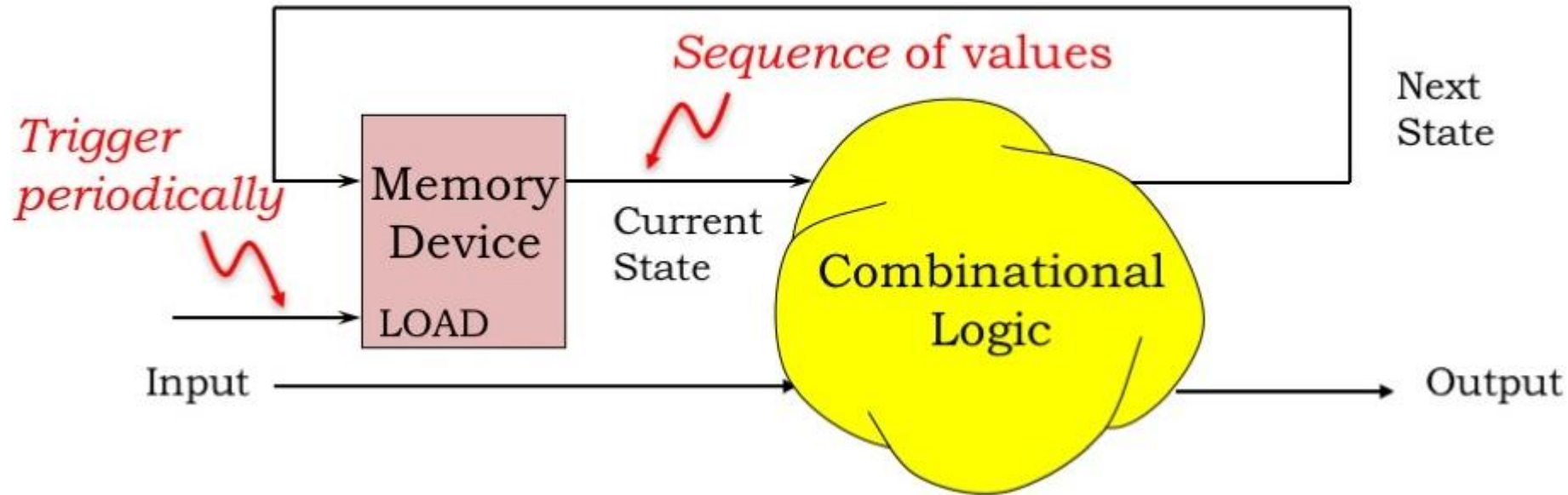
Digital State



Plan: Build a Sequential Circuit with stored digital STATE –

- Memory stores CURRENT state, produced at output
- Combinational Logic computes
 - NEXT state (from input, current state)

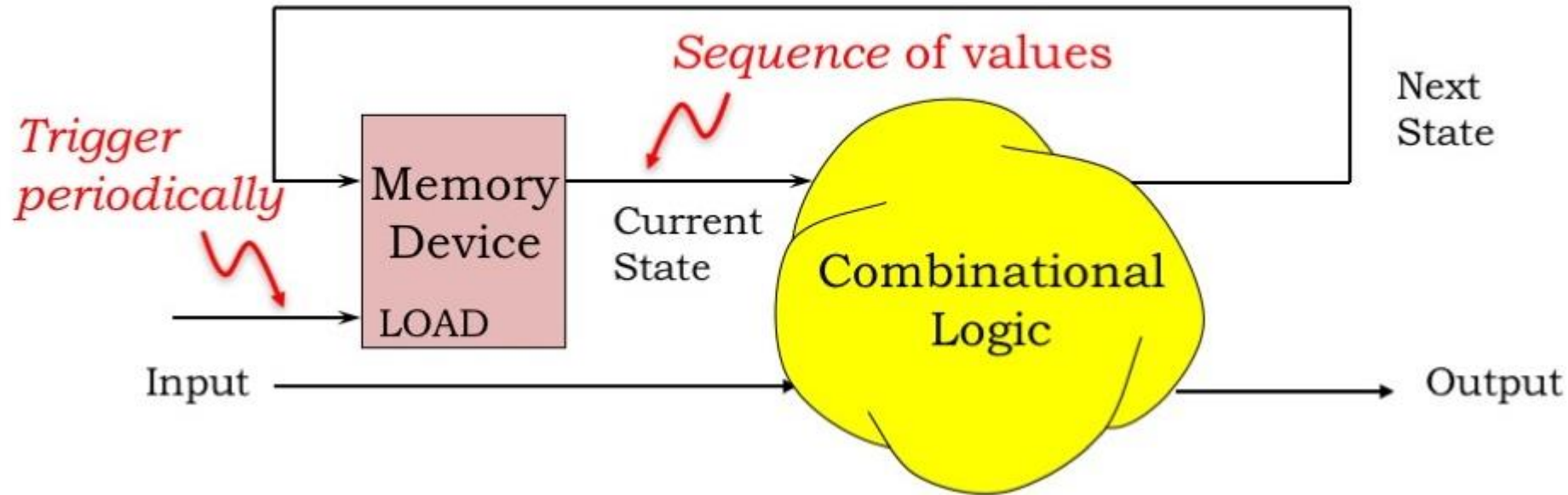
Digital State



Plan: Build a Sequential Circuit with stored digital STATE –

- Memory stores CURRENT state, produced at output
- Combinational Logic computes
 - NEXT state (from input, current state)
 - OUTPUT bits (from input, current state)

Digital State

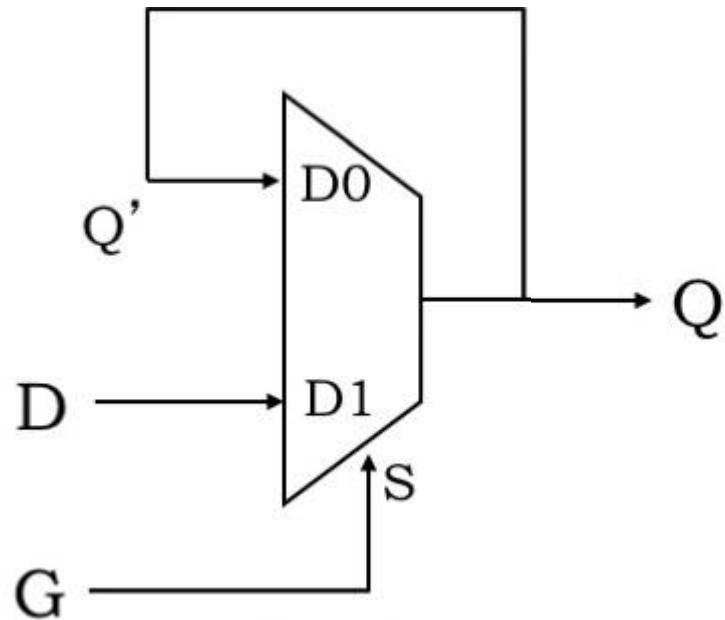


Plan: Build a Sequential Circuit with stored digital STATE –

- Memory stores CURRENT state, produced at output
- Combinational Logic computes
 - NEXT state (from input, current state)
 - OUTPUT bits (from input, current state)
- State changes on LOAD control input

Settable memory element (D-latch)

It's easy to build a settable storage element (called a **latch**) using a *lenient* MUX:



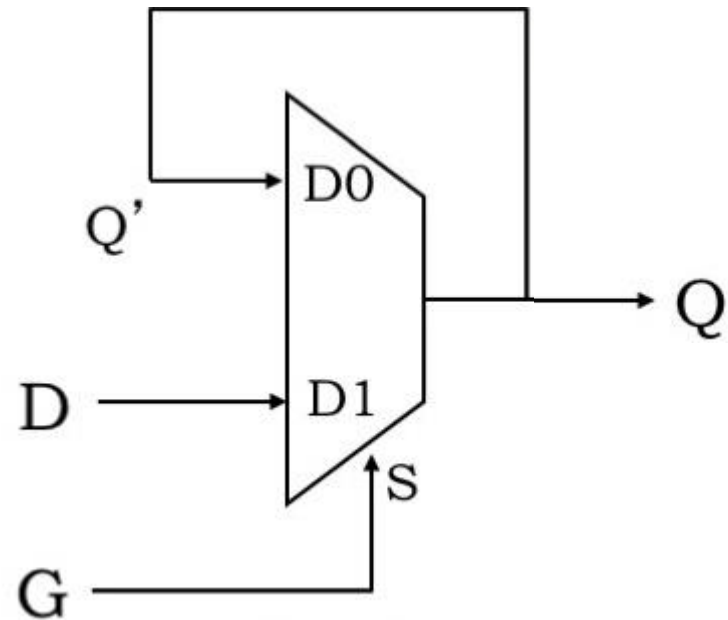
D: data input

G: gate input

Q: state output

Settable memory element (D-latch)

It's easy to build a settable storage element (called a **latch**) using a *lenient* MUX:

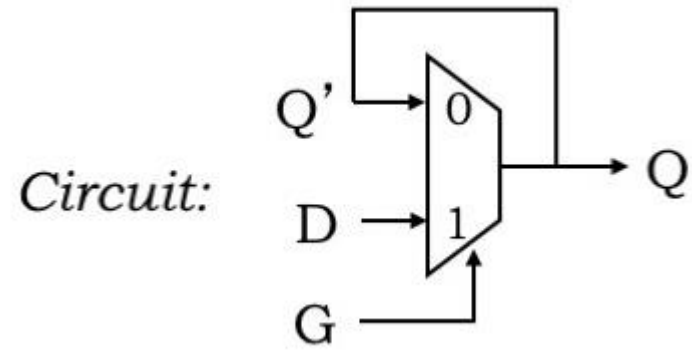


D: data input
G: gate input
Q: state output

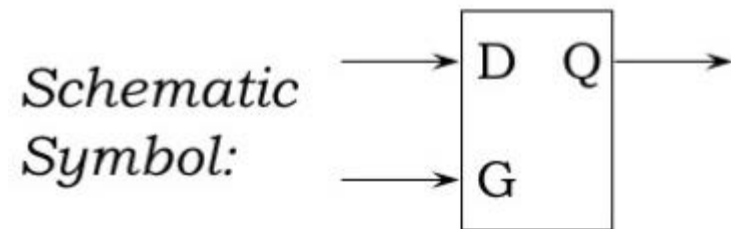
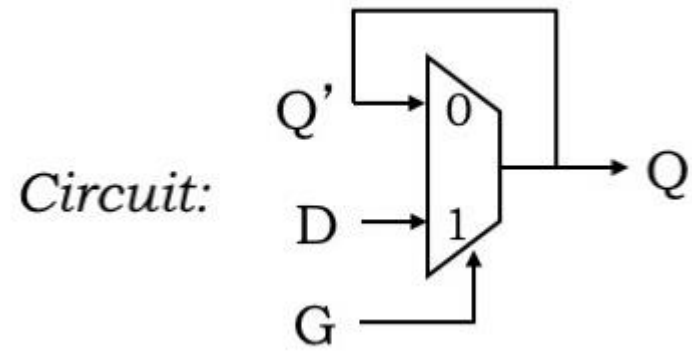
“state” signal appears as both input and output

G	D	Q'	Q	
0	--	0	0	} Q stable
0	--	1	1	
1	0	--	0	} Q follows D
1	1	--	1	

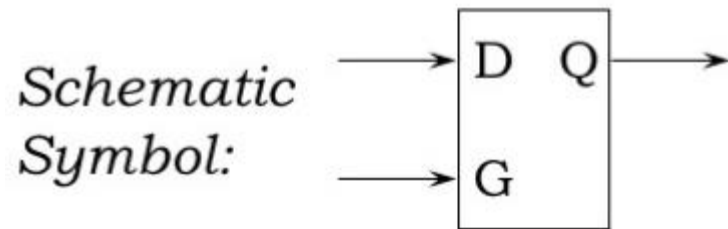
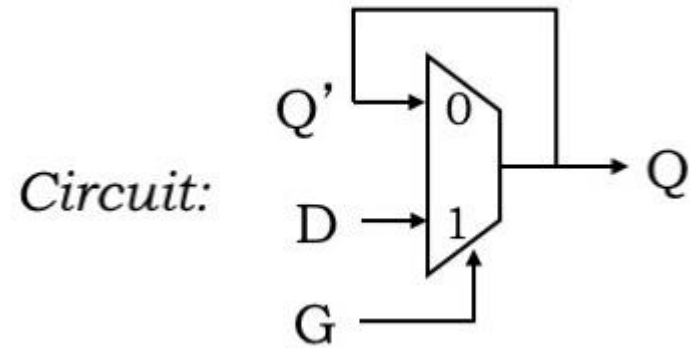
Settable memory element (D-latch)



Settable memory element (D-latch)



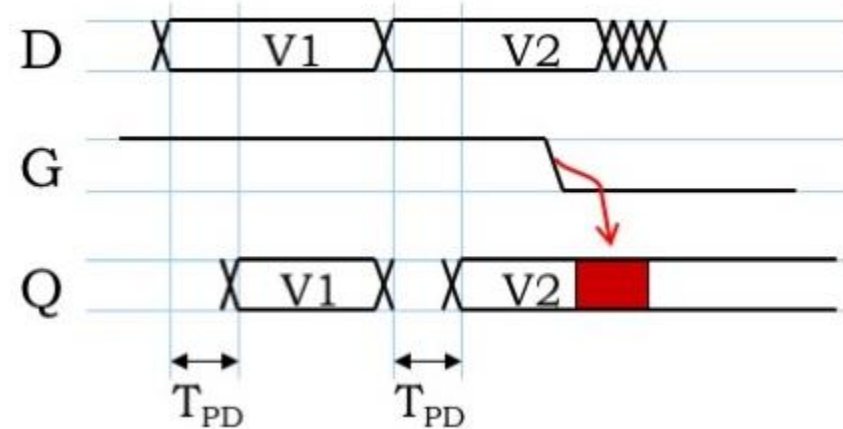
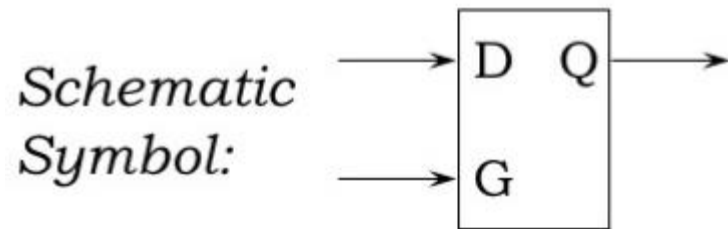
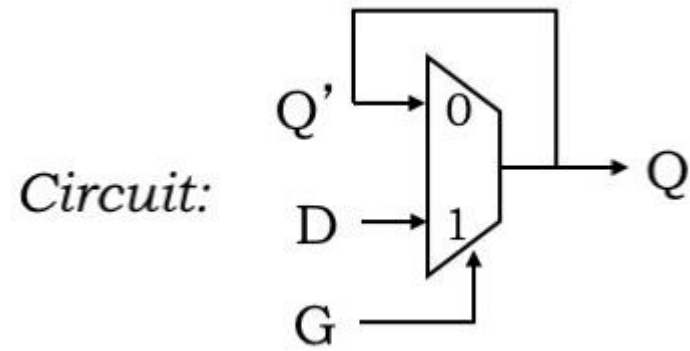
Settable memory element (D-latch)



$G=1$: Q Follows D , *independently of Q'*

$G=0$: Q Holds stable Q' , *independently of D*

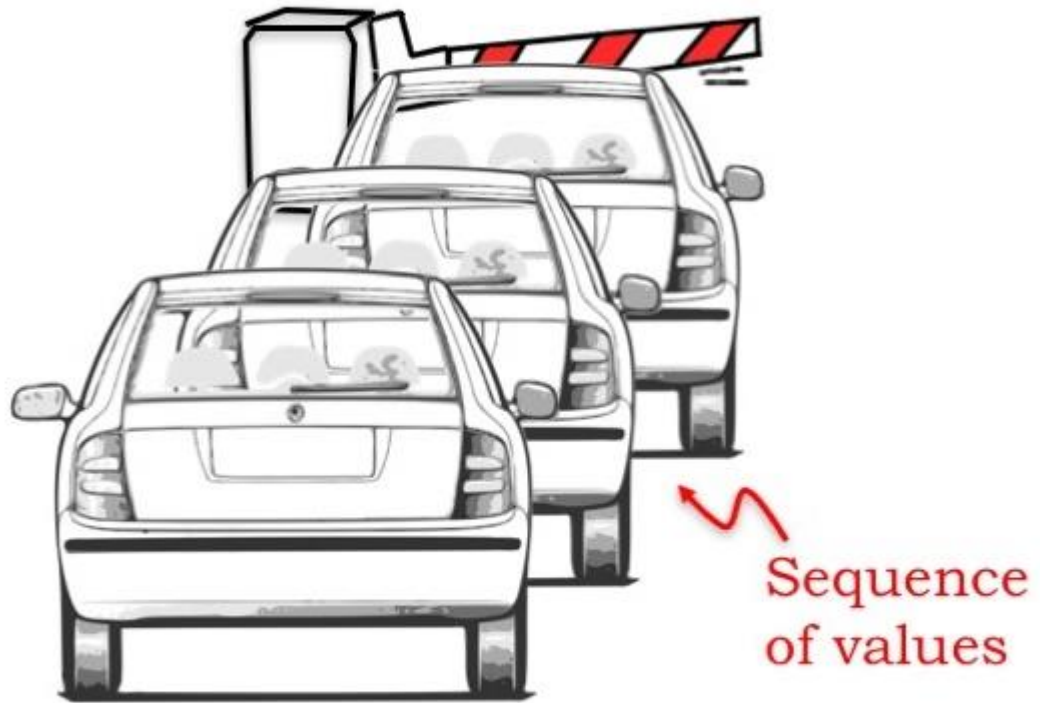
Settable memory element (D-latch)



$G=1$: Q Follows D , *independently of Q'*

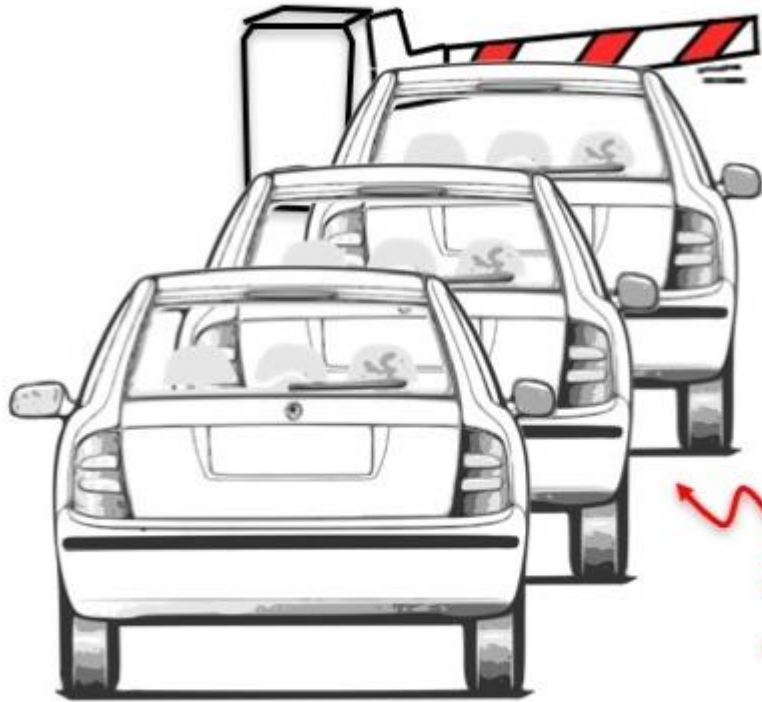
$G=0$: Q Holds stable Q' , *independently of D*

Gate Control System

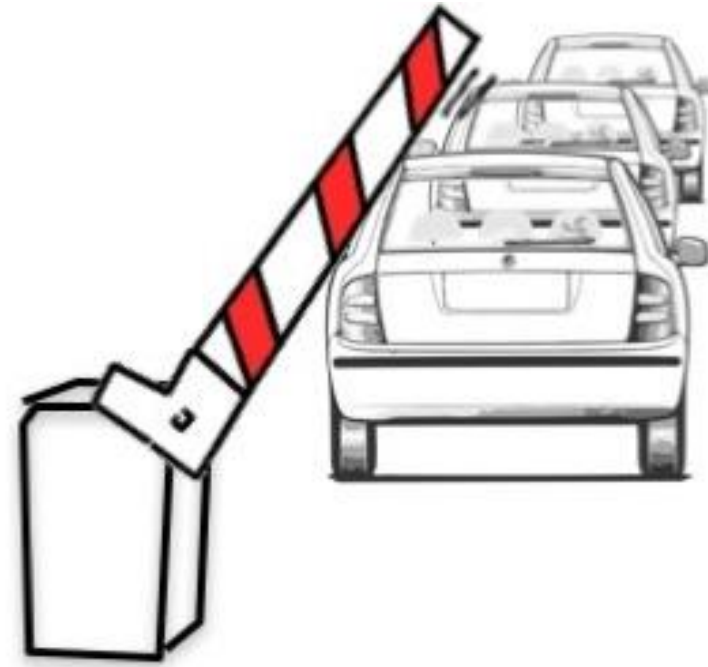


Gate closed

Gate Control System

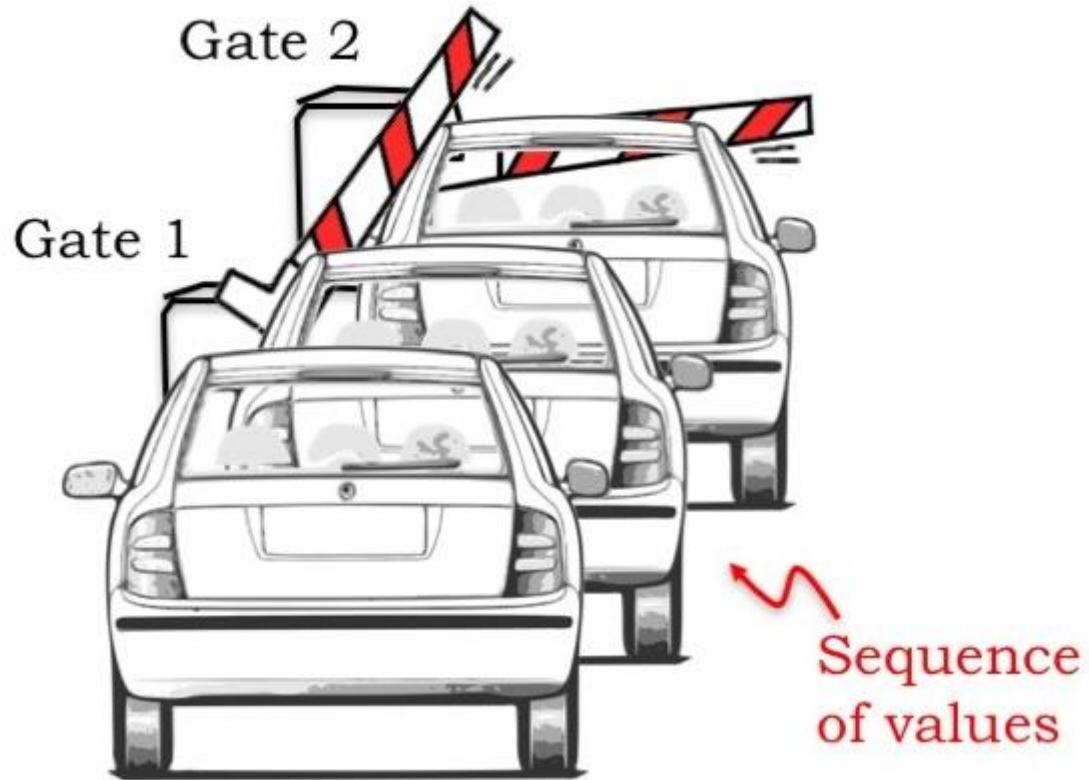


Gate closed



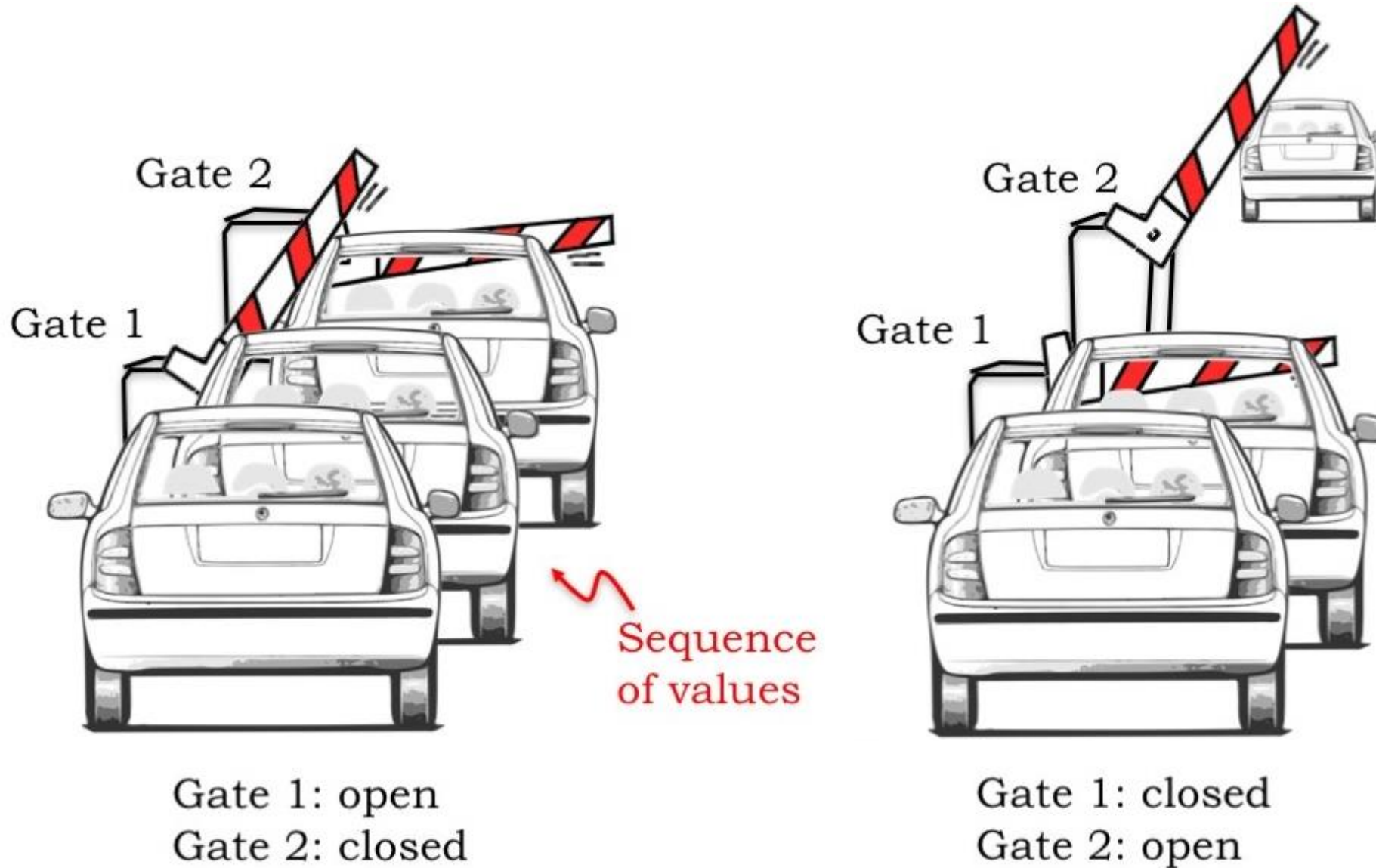
Gate open

Gate Control System (2 Gates)

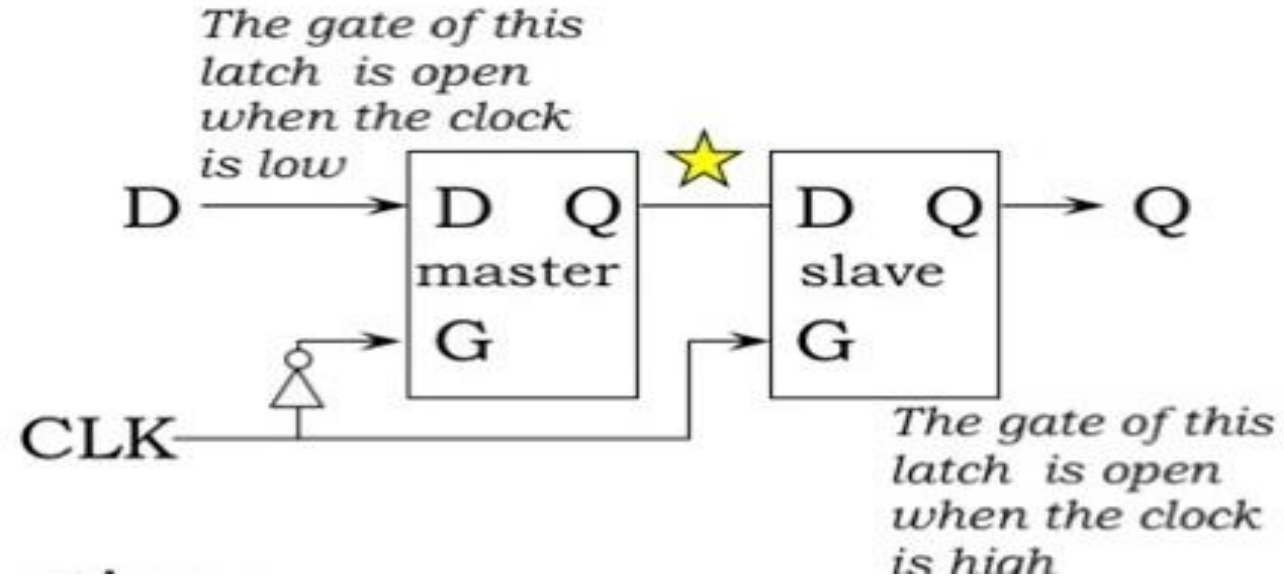


Gate 1: open
Gate 2: closed

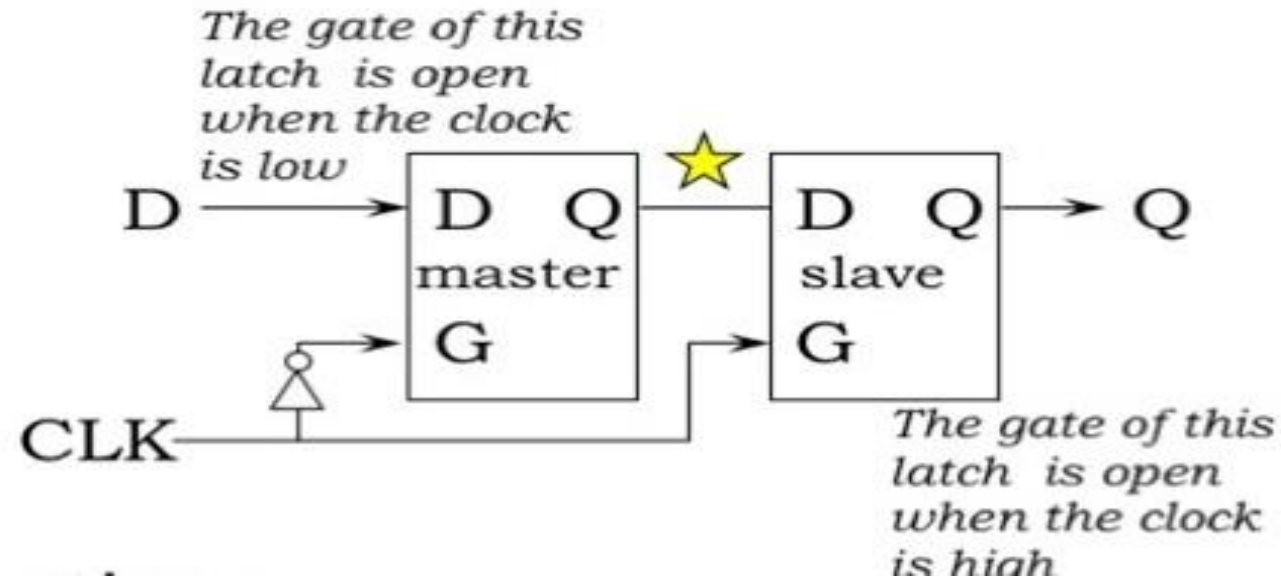
Gate Control System (2 Gates)



Edge-triggered D register

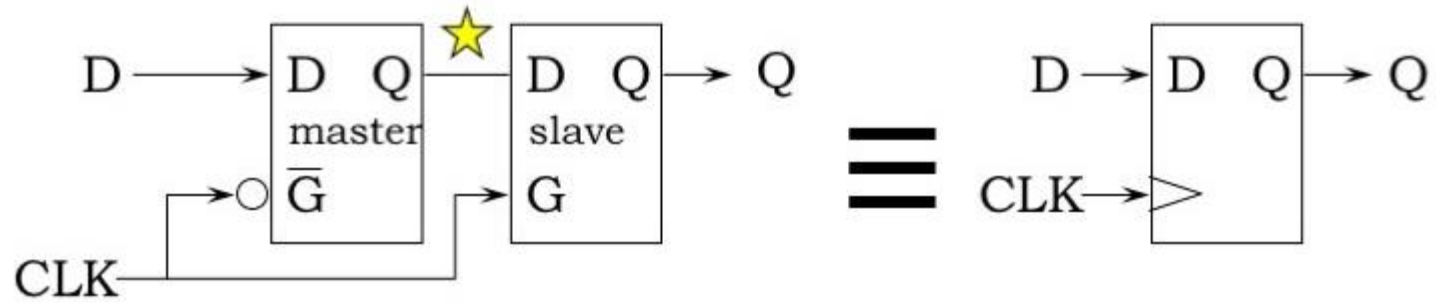


Edge-triggered D register

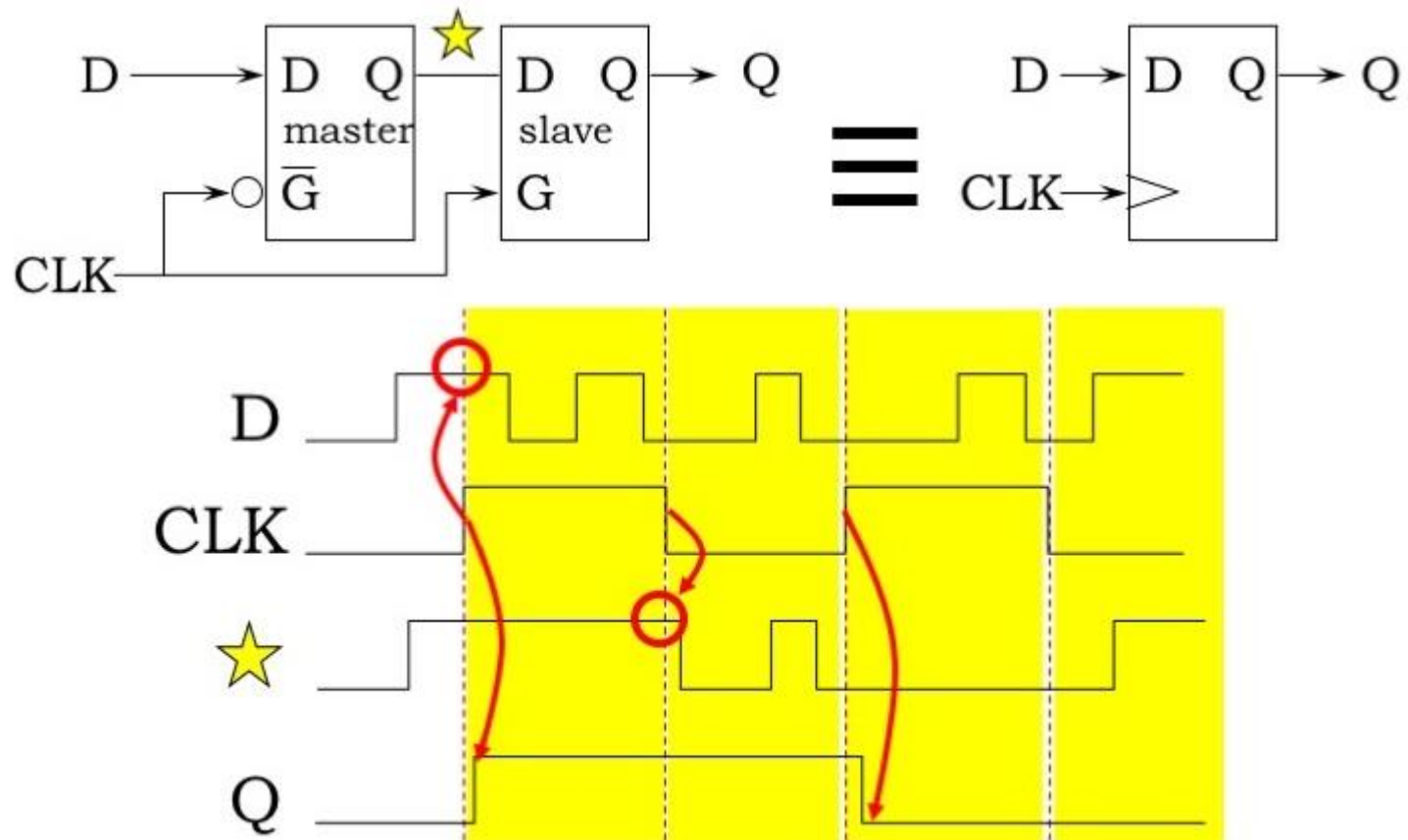


- only one latch “transparent” at any time:
 - master closed when slave is open
 - slave closed when master is open
- ⇒ no combinational path through register
(the feedback path in one of the master or slave latches is always active)

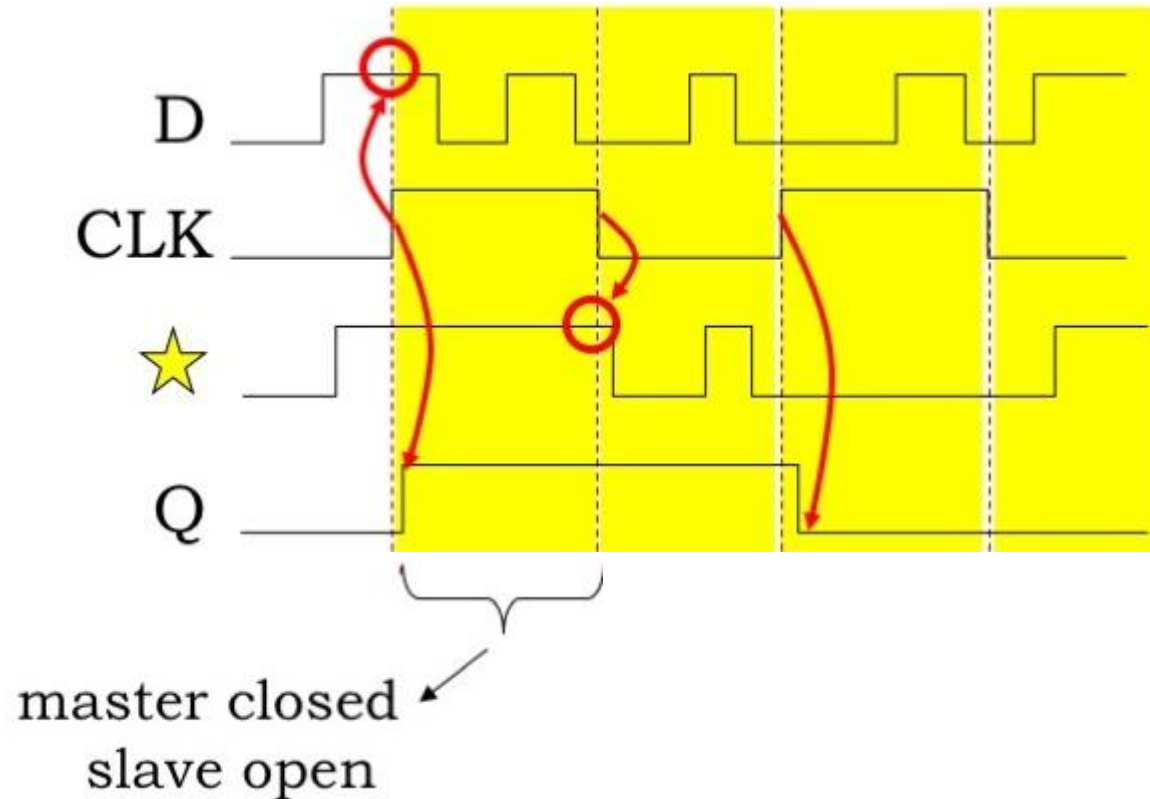
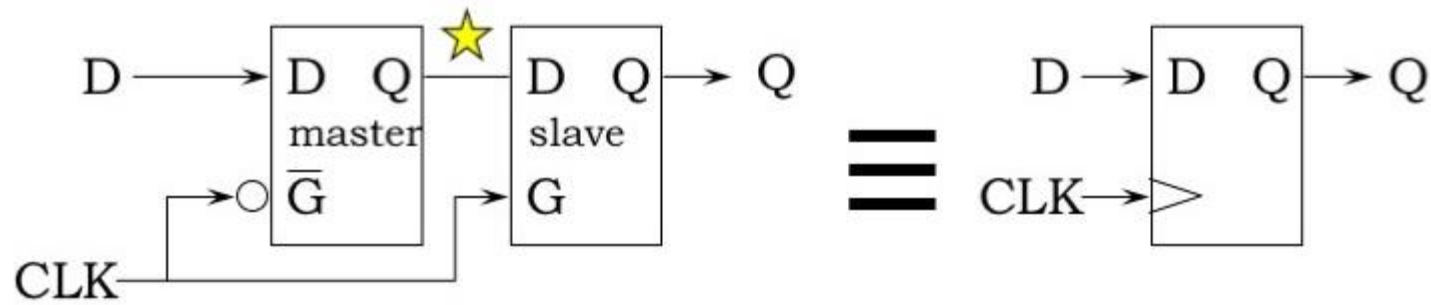
D register wave forms



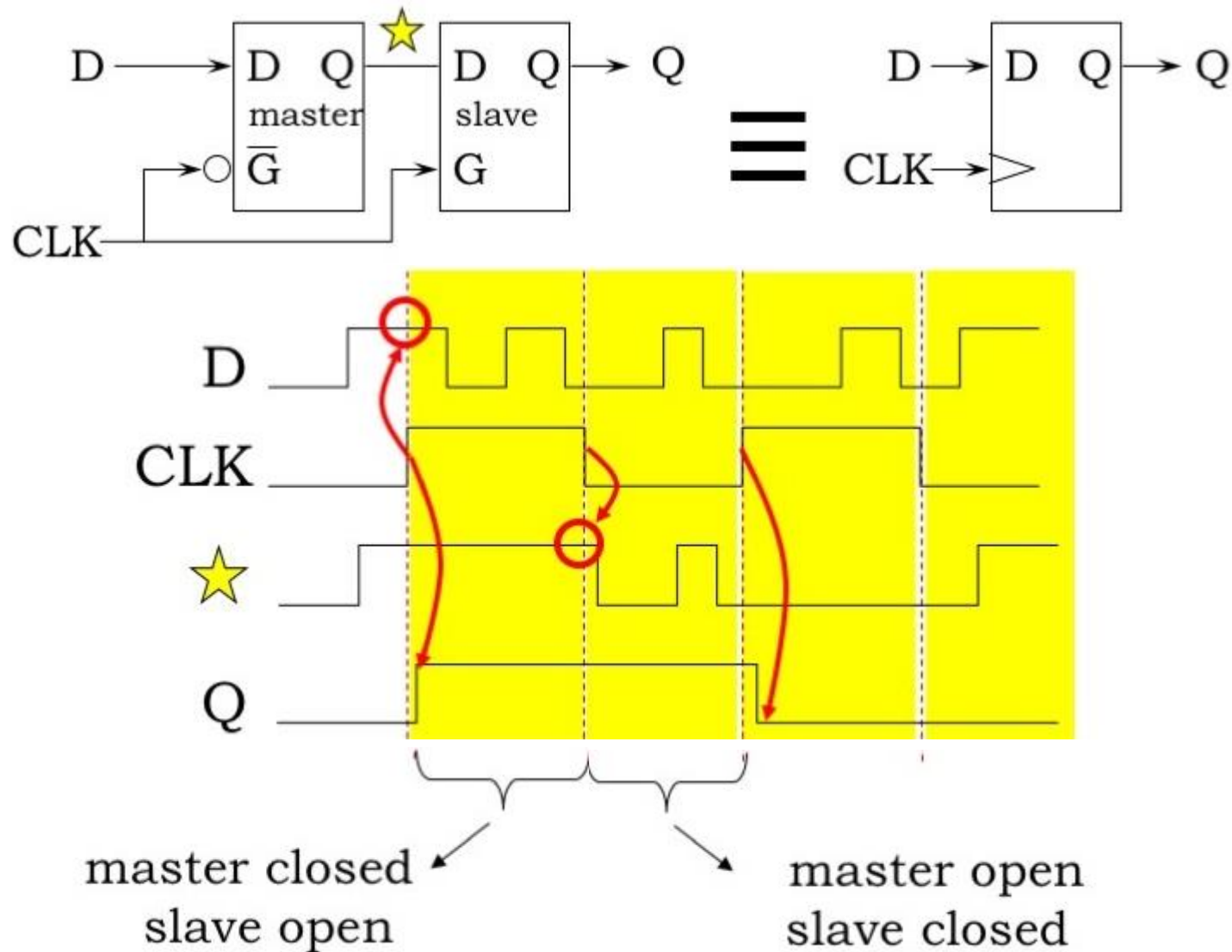
D register wave forms



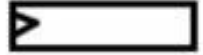
D register wave forms



D register wave forms



Single-clock synchronous circuits

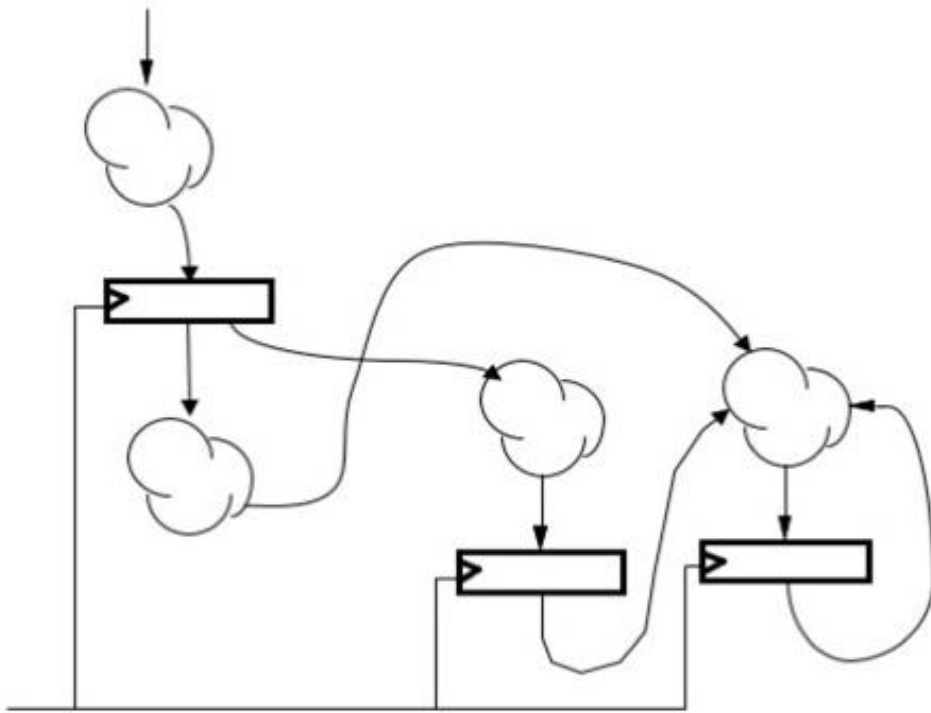


We'll use registers in a highly constrained way to build digital systems:

Single-clock synchronous circuits



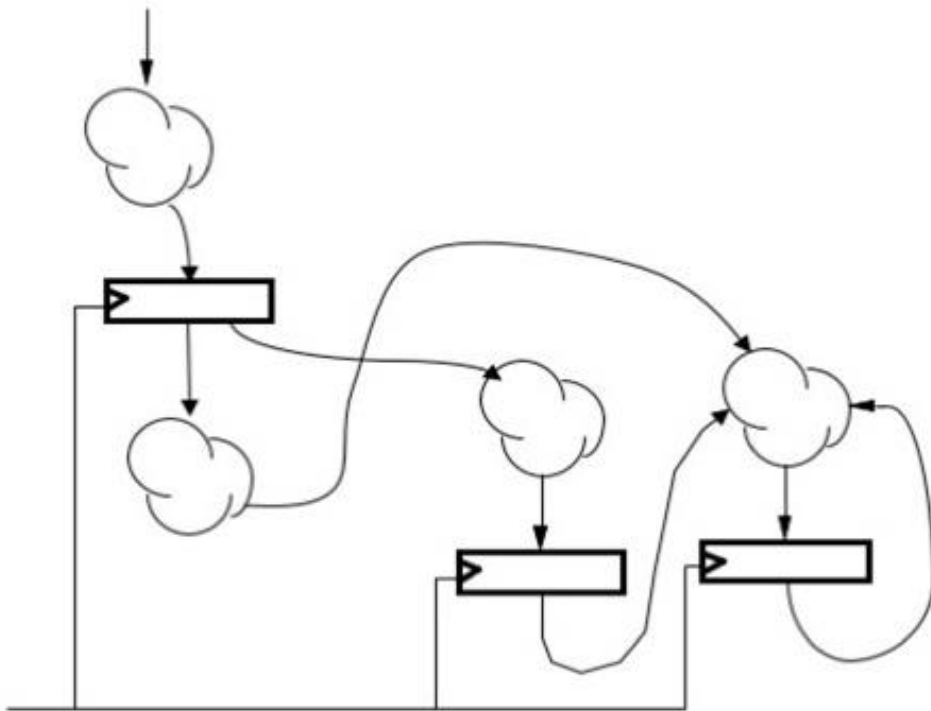
We'll use registers in a highly constrained way to build digital systems:



Single-clock synchronous circuits



We'll use registers in a highly constrained way to build digital systems:



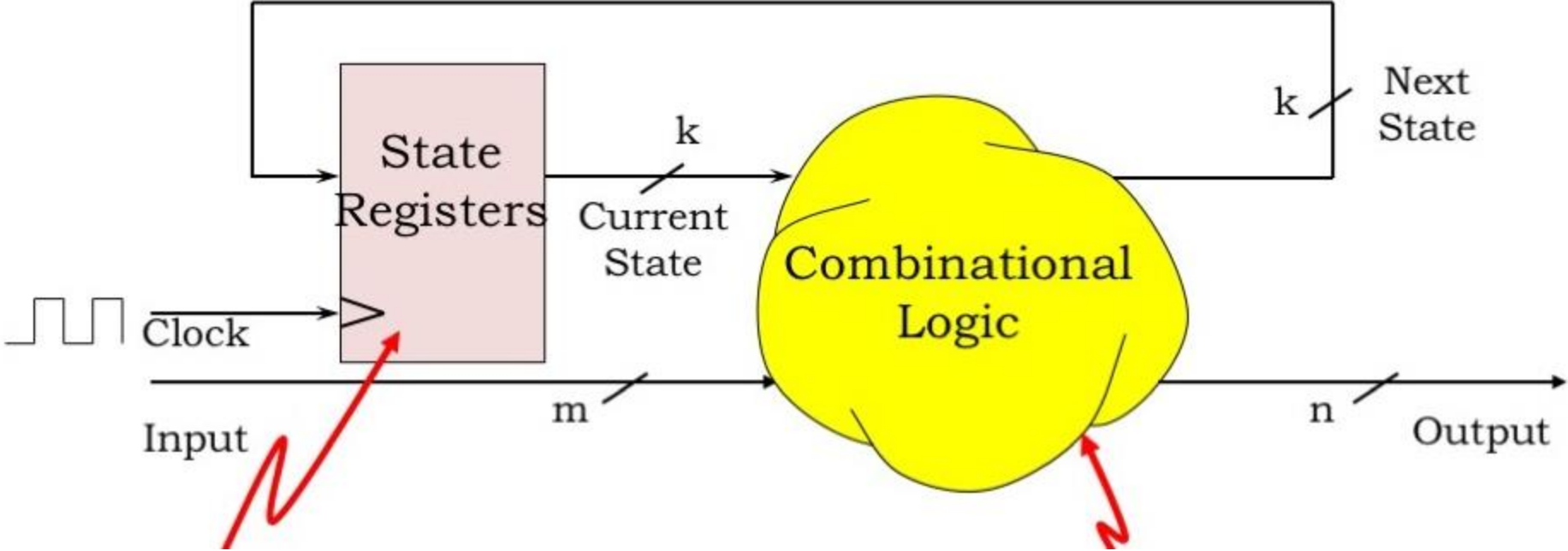
Single-clock Synchronous Discipline

- No combinational cycles
- Single periodic clock signal shared among all clocked devices
- Only care about value of register data inputs just before rising edge of clock

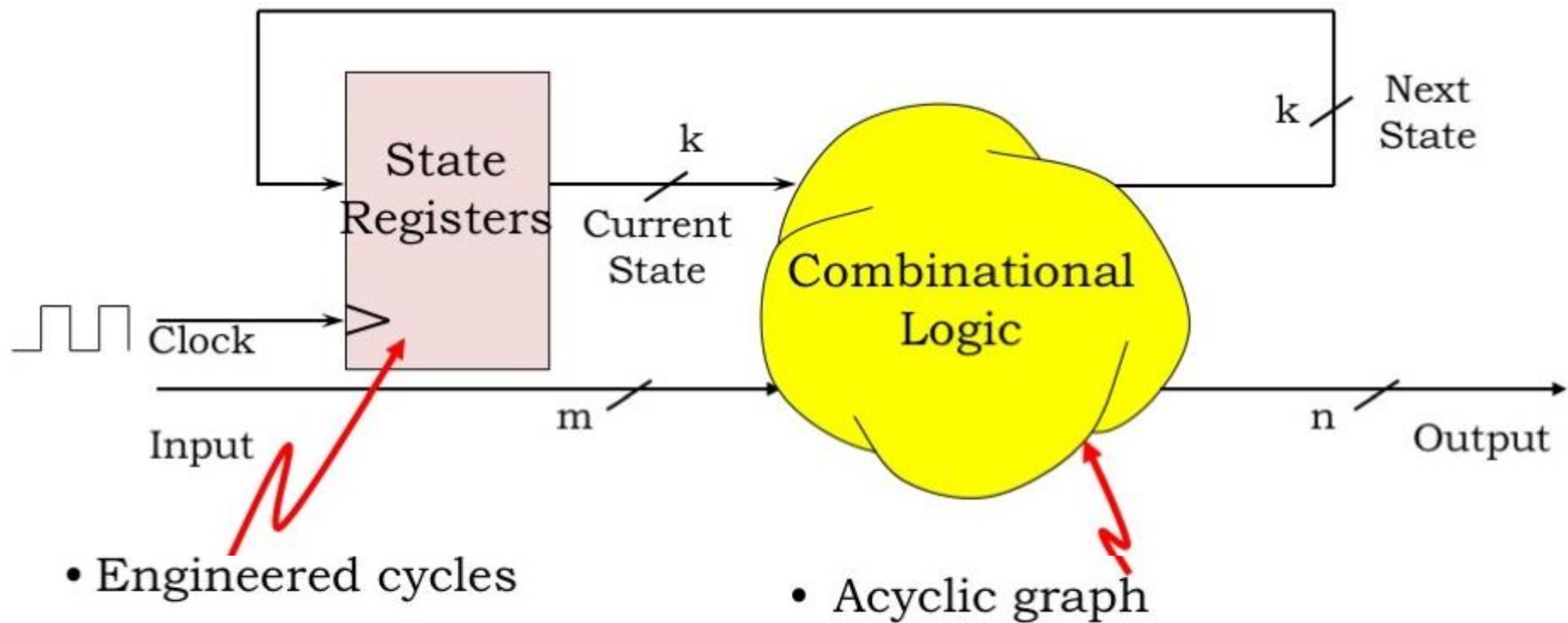
Finite State Machine

Computer Architecture

New Machine

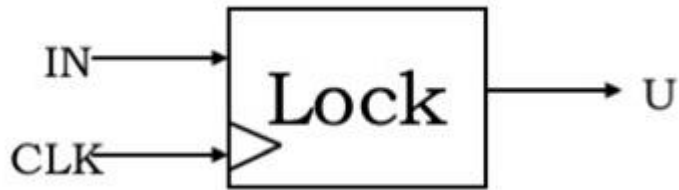


New Machine



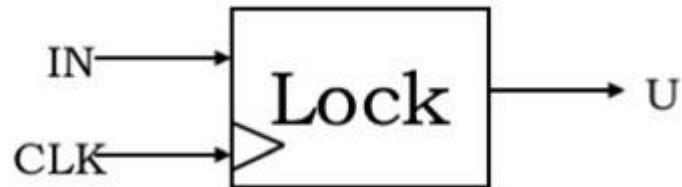
Sequential Combination Lock

Lets make a digital binary *Combination Lock*:



Sequential Combination Lock

Lets make a digital binary *Combination Lock*:

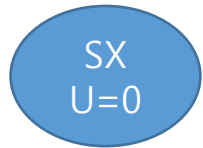


Specification:

- One input (“0” or “1”)
- One output (“Unlock” signal)
- UNLOCK is 1 if and only if:

Last 4 inputs were the
“combination”: 0110

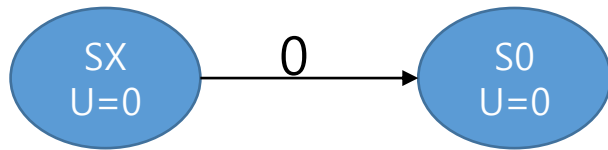
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

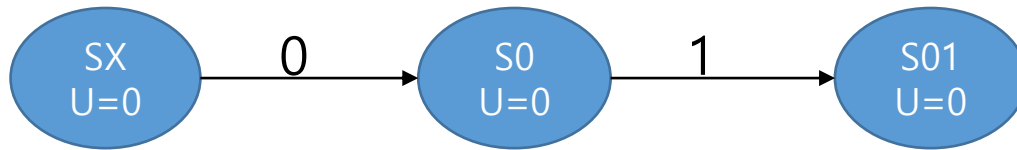
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

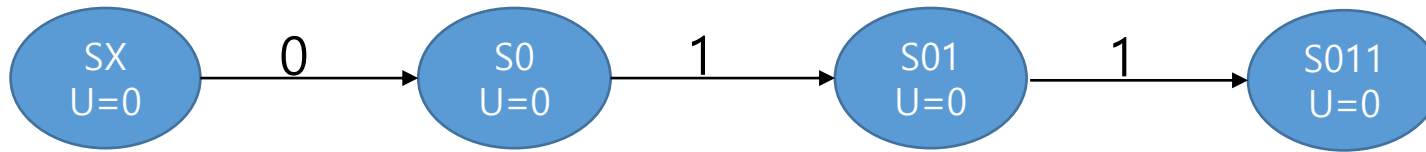
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

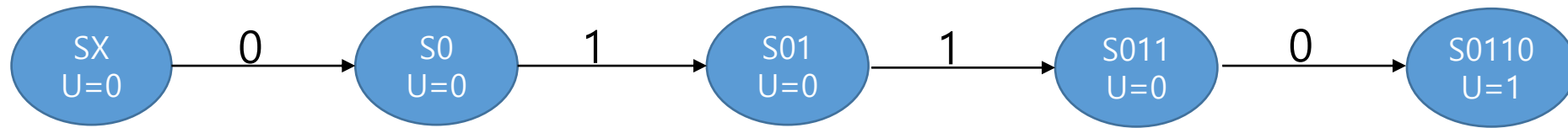
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

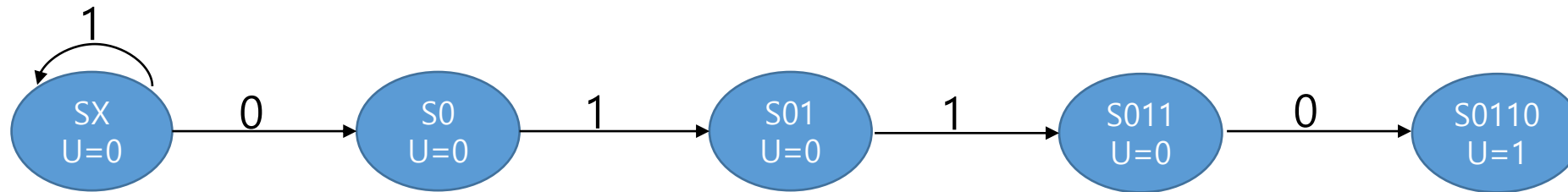
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

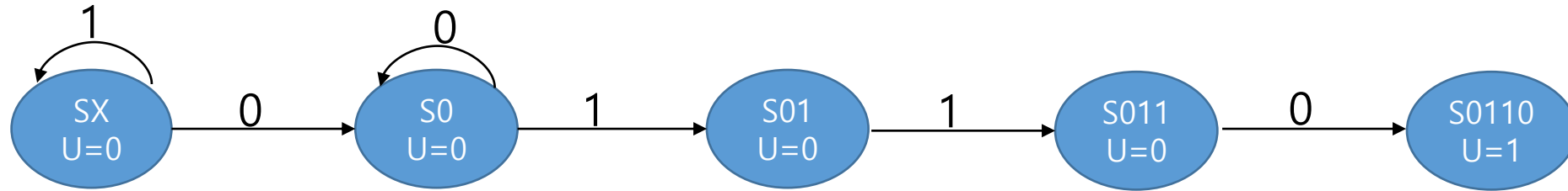
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

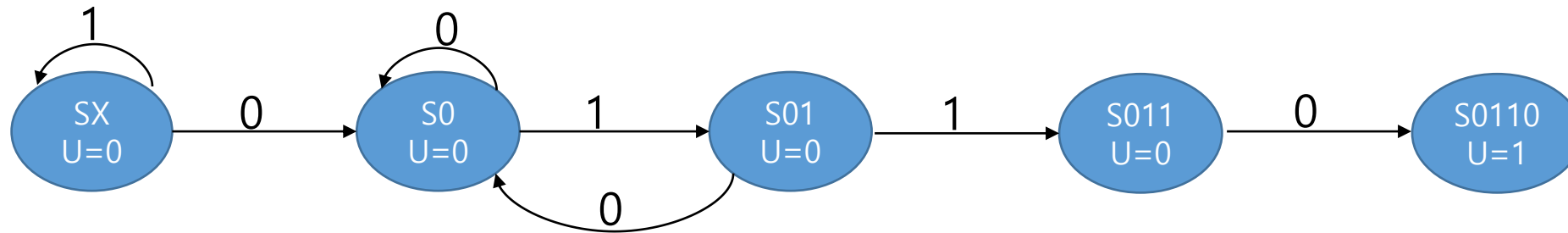
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

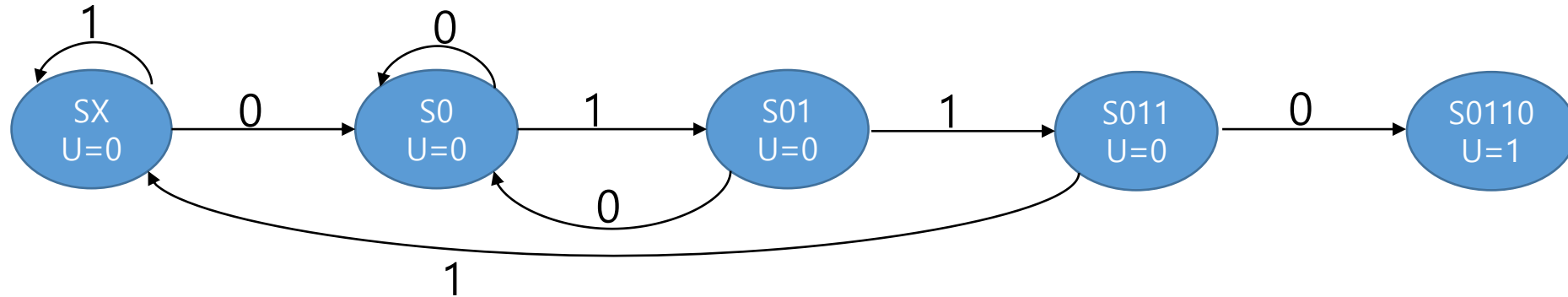
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

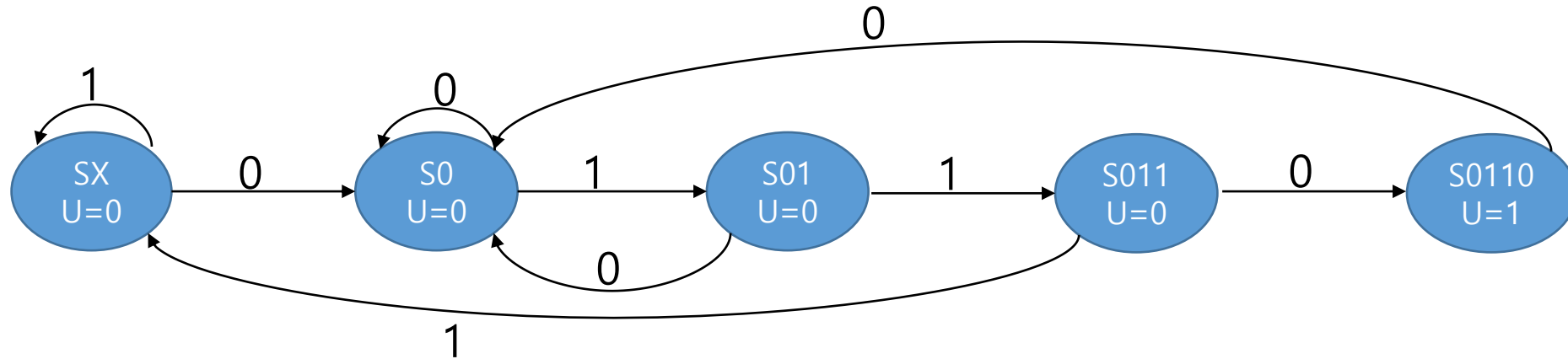
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

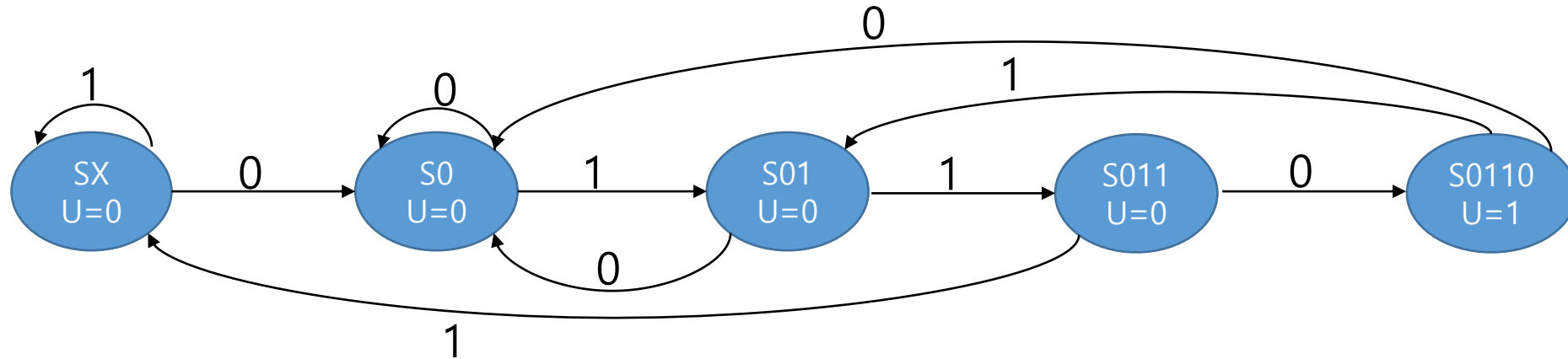
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

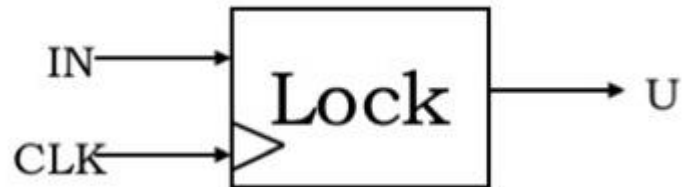
State transition diagram



Designing our lock ...

- Need an initial state; call it SX.
- Must have a separate state for each step of the proper entry sequence
- Must handle other (erroneous) entries

What if?

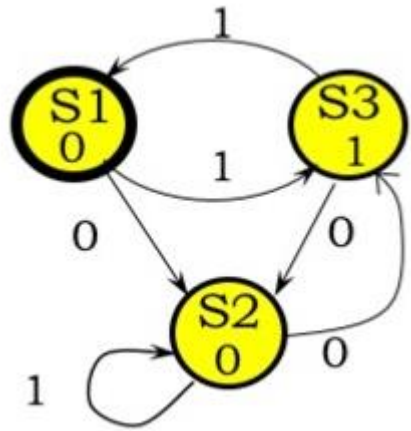


Specification:

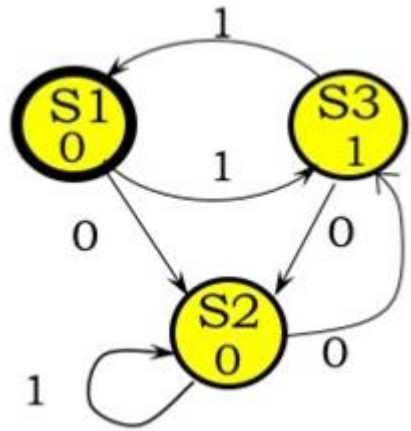
- One input (“0” to “9”)
- One output (“Unlock” signal)
- UNLOCK is 1 if and only if:

Last 4 inputs were the
“combination”: 1379

Valid state diagram



Valid state diagram

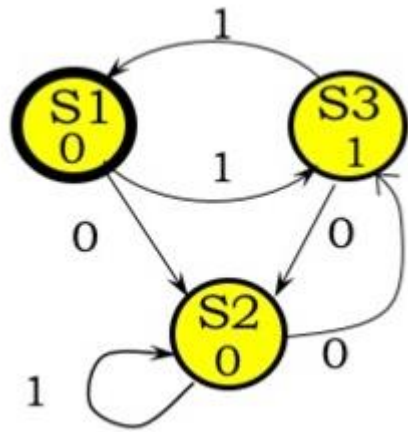


Arcs leaving a state must be:

(1) **mutually exclusive**

– *can't have two choices for a given input value*

Valid state diagram



Arcs leaving a state must be:

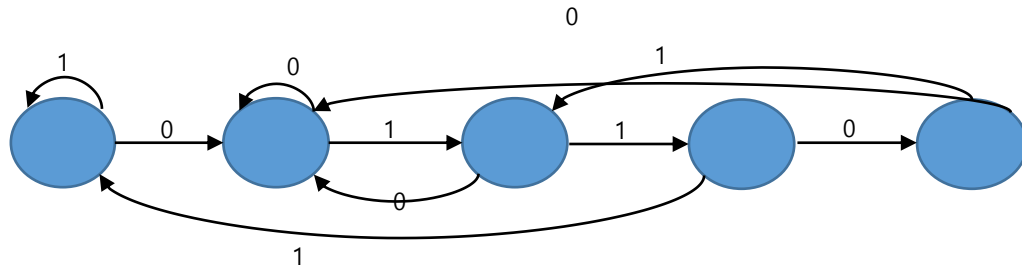
(1) **mutually exclusive**

- *can't have two choices for a given input value*

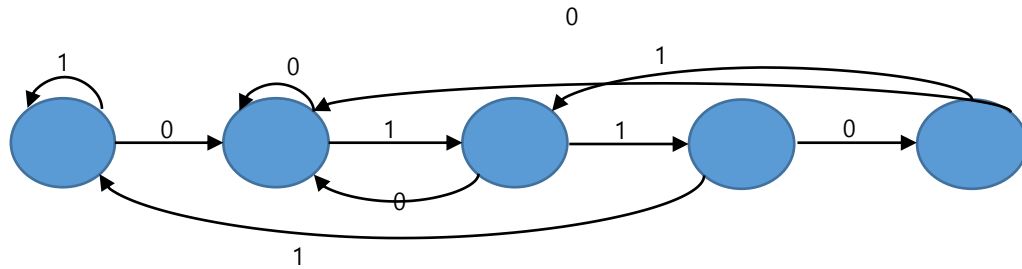
(2) **collectively exhaustive**

- *every state must specify what happens for each possible input combination. "Nothing happens" means arc back to itself.*

Truth table for transition diagram



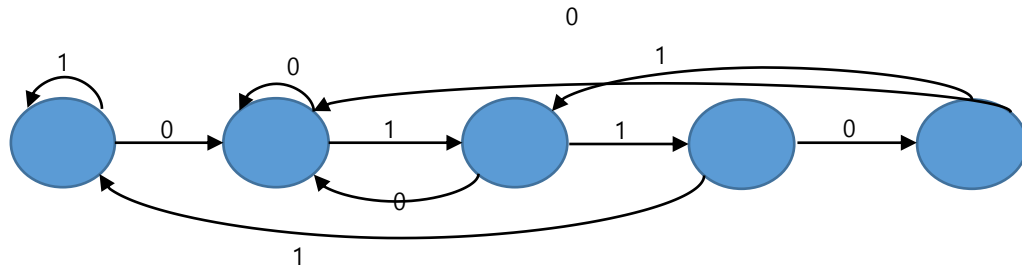
Truth table for transition diagram



All state transition diagrams can be described by truth tables...

IN Current State		Next State Unlock	
0	SX	S0	0
1	SX	SX	0
0	S0	S0	0
1	S0	S01	0
0	S01	S0	0
1	S01	S011	0
0	S011	S0110	0
1	S011	SX	0
0	S0110	S0	1
1	S0110	S01	1

Truth table for transition diagram

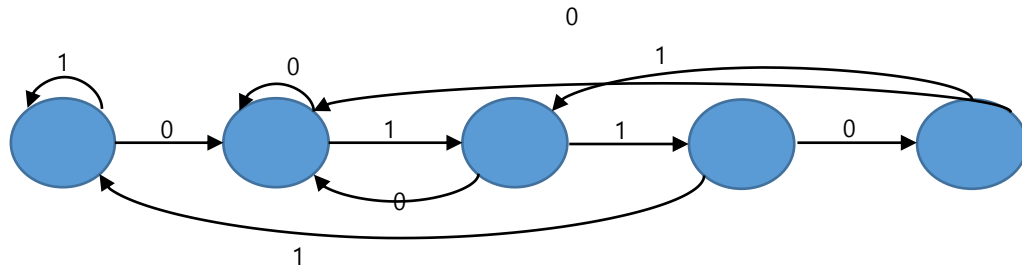


All state transition diagrams can be described by truth tables...

Binary encodings are assigned to each state (a bit of an art)

IN Current State		Next State Unlock	
0	SX	S0	0
1	SX	SX	0
0	S0	S0	0
1	S0	S01	0
0	S01	S0	0
1	S01	S011	0
0	S011	S0110	0
1	S011	SX	0
0	S0110	S0	1
1	S0110	S01	1

Truth table for transition diagram

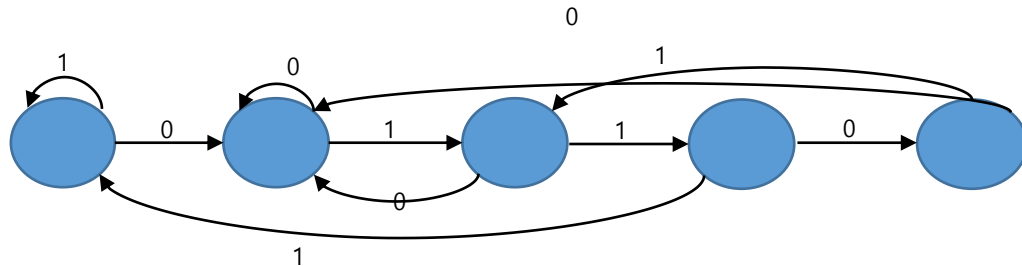


All state transition diagrams can be described by truth tables...

Binary encodings are assigned to each state (a bit of an art)

IN Current State			Next State Unlock		
0	000	SX	S0	001	0
1	000	SX	SX	000	0
0	001	S0	S0	001	0
1	001	S0	S01	011	0
0	011	S01	S0	001	0
1	011	S01	S011	010	0
0	010	S011	S0110	100	0
1	010	S011	SX	000	0
0	100	S0110	S0	001	1
1	100	S0110	S01	011	1

Truth table for transition diagram



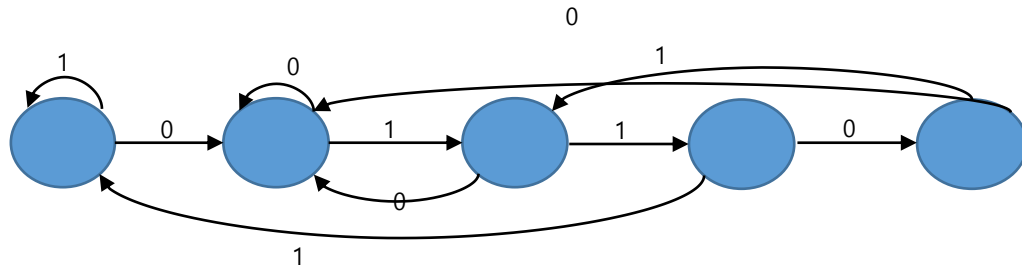
All state transition diagrams can be described by truth tables...

Binary encodings are assigned to each state (a bit of an art)

The truth table can then be simplified using the reduction techniques we learned for combinational logic

IN Current State			Next State			Unlock
0	000	SX	S0	001	0	
1	000	SX	SX	000	0	
0	001	S0	S0	001	0	
1	001	S0	S01	011	0	
0	011	S01	S0	001	0	
1	011	S01	S011	010	0	
0	010	S011	S0110	100	0	
1	010	S011	SX	000	0	
0	100	S0110	S0	001	1	
1	100	S0110	S01	011	1	

Truth table for transition diagram



All state transition diagrams can be described by truth tables...

Binary encodings are assigned to each state (a bit of an art)

The truth table can then be simplified using the reduction techniques we learned for combinational logic

IN Current State			Next State Unlock		
0	000	SX	S0	001	0
1	000	SX	SX	000	0
0	001	S0	S0	001	0
1	001	S0	S01	011	0
0	011	S01	S0	001	0
1	011	S01	S011	010	0
0	010	S011	S0110	100	0
1	010	S011	SX	000	0
0	100	S0110	S0	001	1
1	100	S0110	S01	011	1

How to implement this?

Using Truth Table

A	B	C	In	A'	B'	C'	Out
0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	1	0
0	0	1	1	0	1	1	0
0	1	1	0	0	0	1	0
0	1	1	1	0	1	0	0
0	1	0	0	1	0	0	0
0	1	0	1	0	0	0	0
1	0	0	0	0	0	1	1
1	0	0	1	0	1	1	1

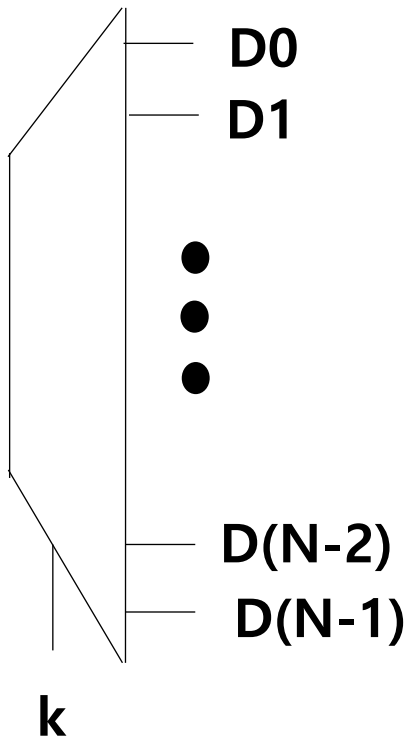
Using Truth Table

A	B	C	In	A'	B'	C'	Out
0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	1	0
0	0	1	1	0	1	1	0
0	1	1	0	0	0	1	0
0	1	1	1	0	1	0	0
0	1	0	0	1	0	0	0
0	1	0	1	0	0	0	0
1	0	0	0	0	0	1	1
1	0	0	1	0	1	1	1

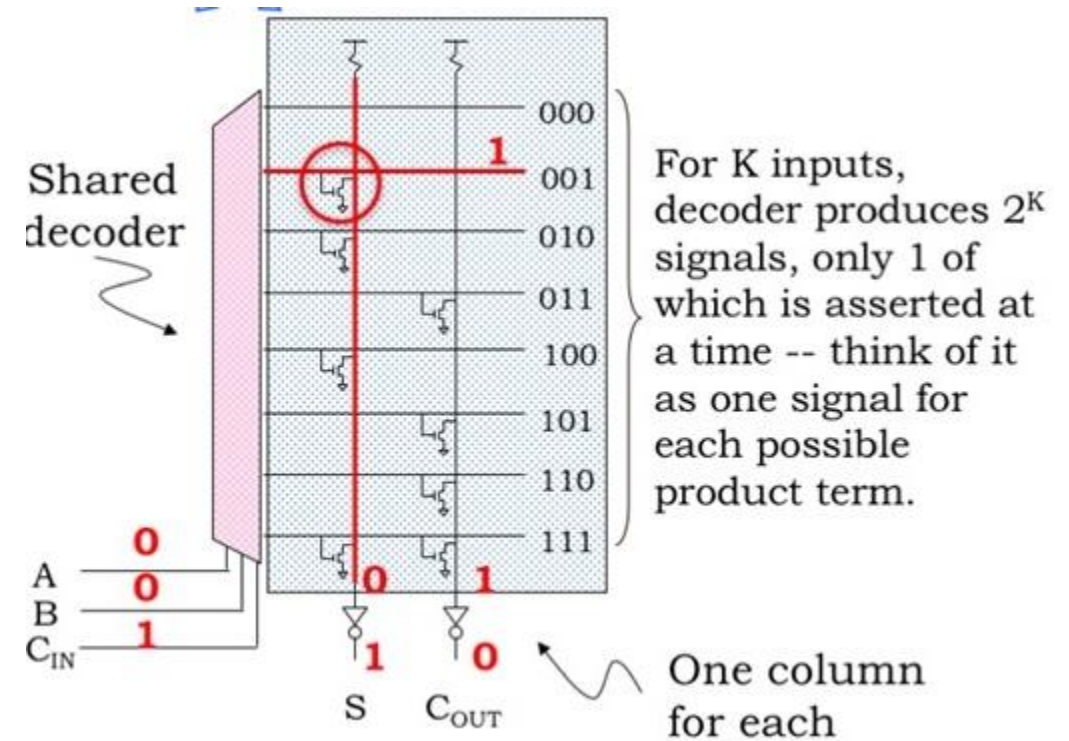
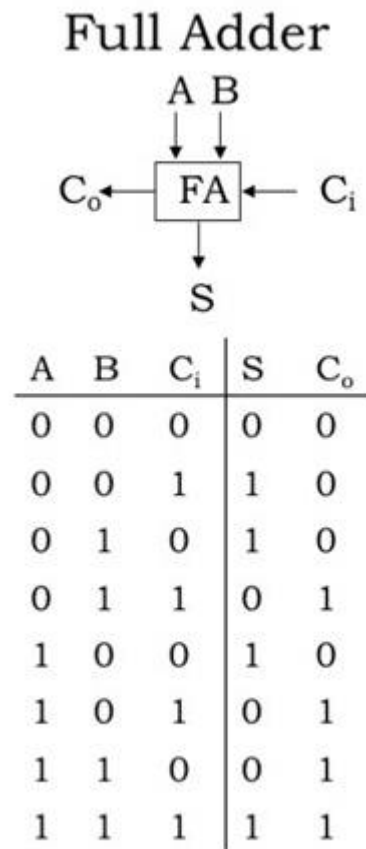
- **A' = ?**
- **B' = ?**
- **C' = ?**
- **Out = ?**

Decoders and ROM

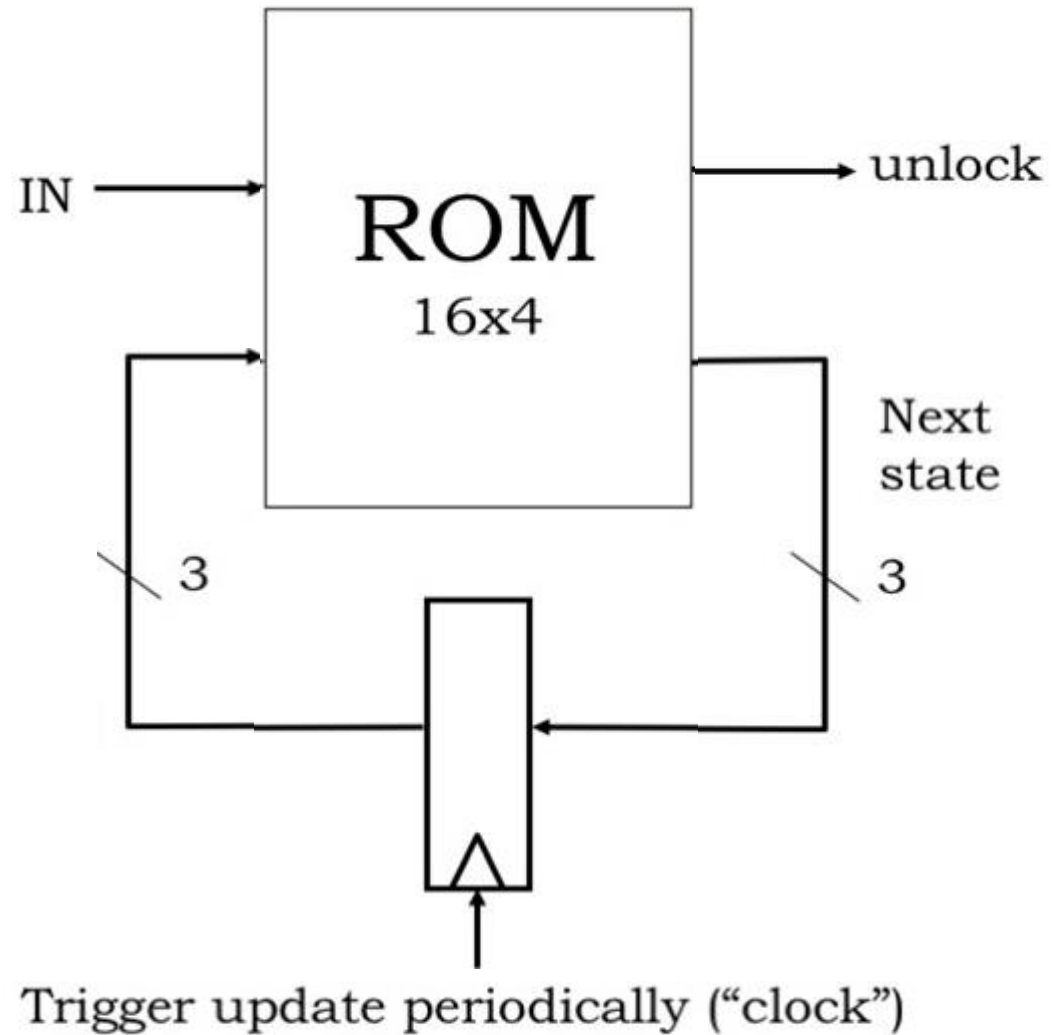
- Decoder
 - k select inputs
 - $N=2^k$ outputs
 - One of D s is HIGH



- Read Only Memory (ROM)



Now put it in a ROM



Programmable Machines

Computer Architecture

Example: Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$

```
a = 1;  
b = N;  
do {  
    a = a * b;  
    b = b - 1;  
} while (b != 0)
```

Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$

```
a = 1;  
b = N;  
do {  
    a = a * b;  
    b = b - 1;  
} while (b != 0)
```

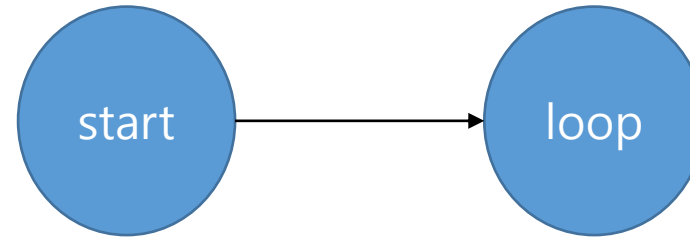


```
a ← 1  
b ← N
```

Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$

```
a = 1;  
b = N;  
do {  
    a = a * b;  
    b = b - 1;  
} while (b != 0)
```



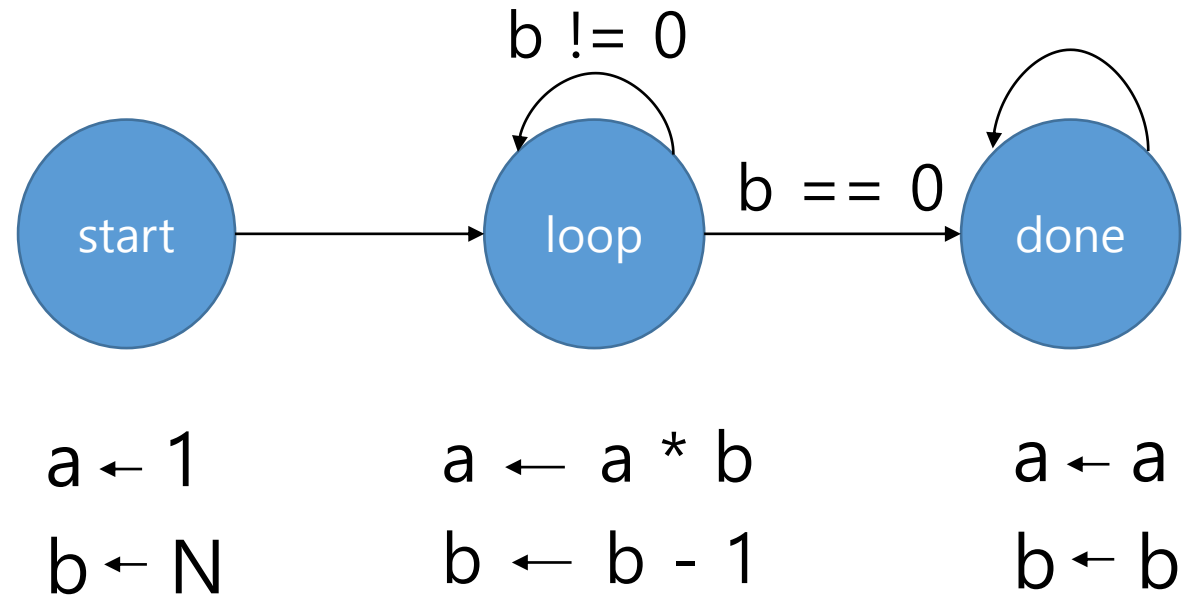
$a \leftarrow 1$
 $b \leftarrow N$

$a \leftarrow a * b$
 $b \leftarrow b - 1$

Factorial

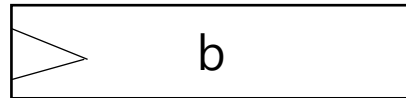
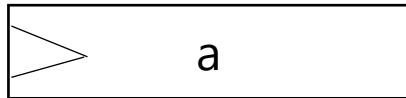
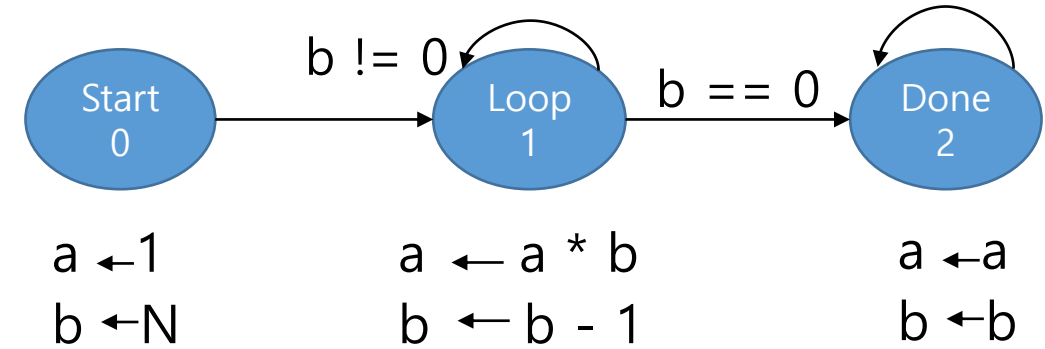
$$N! = N * (N-1) * (N-2) * \dots * 1$$

```
a = 1;  
b = N;  
do {  
    a = a * b;  
    b = b - 1;  
} while (b != 0)
```



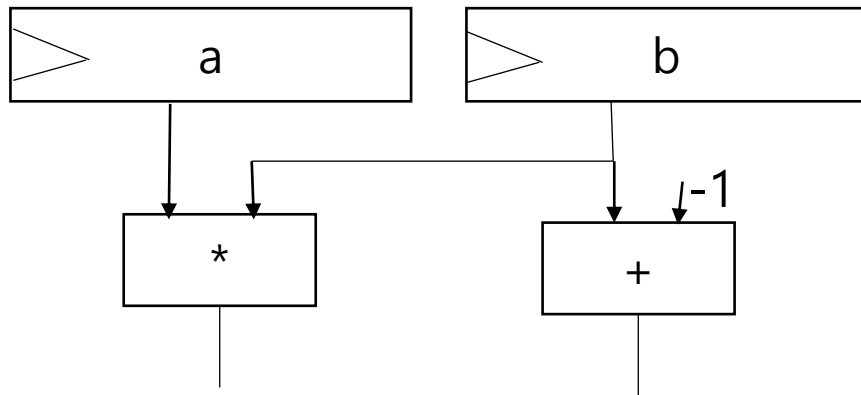
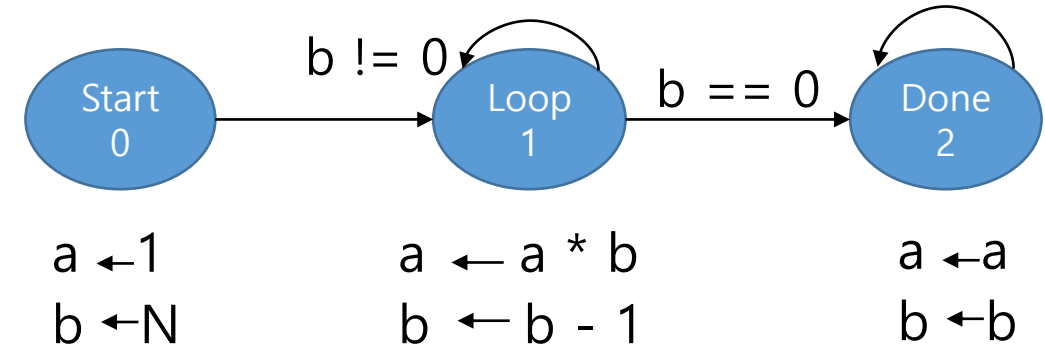
Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$



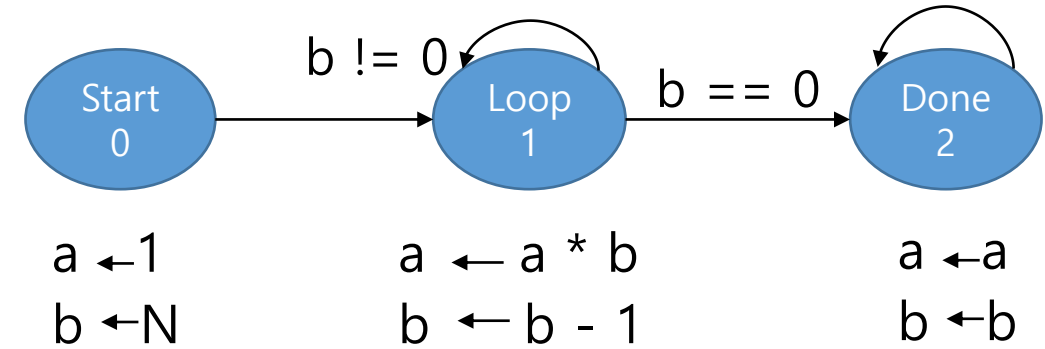
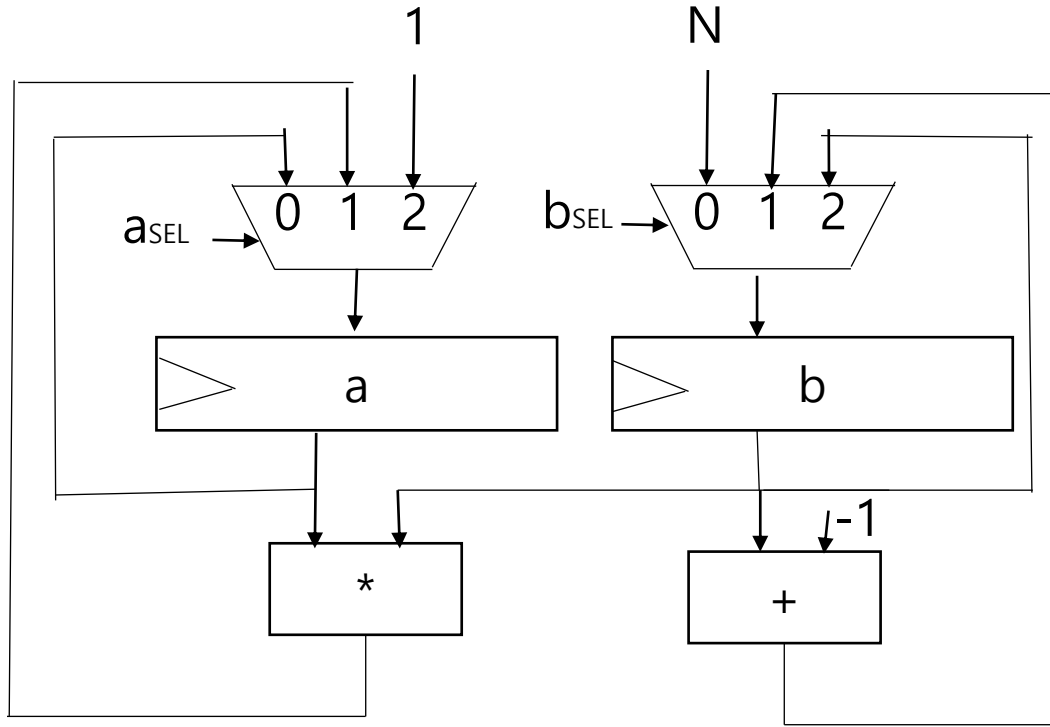
Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$



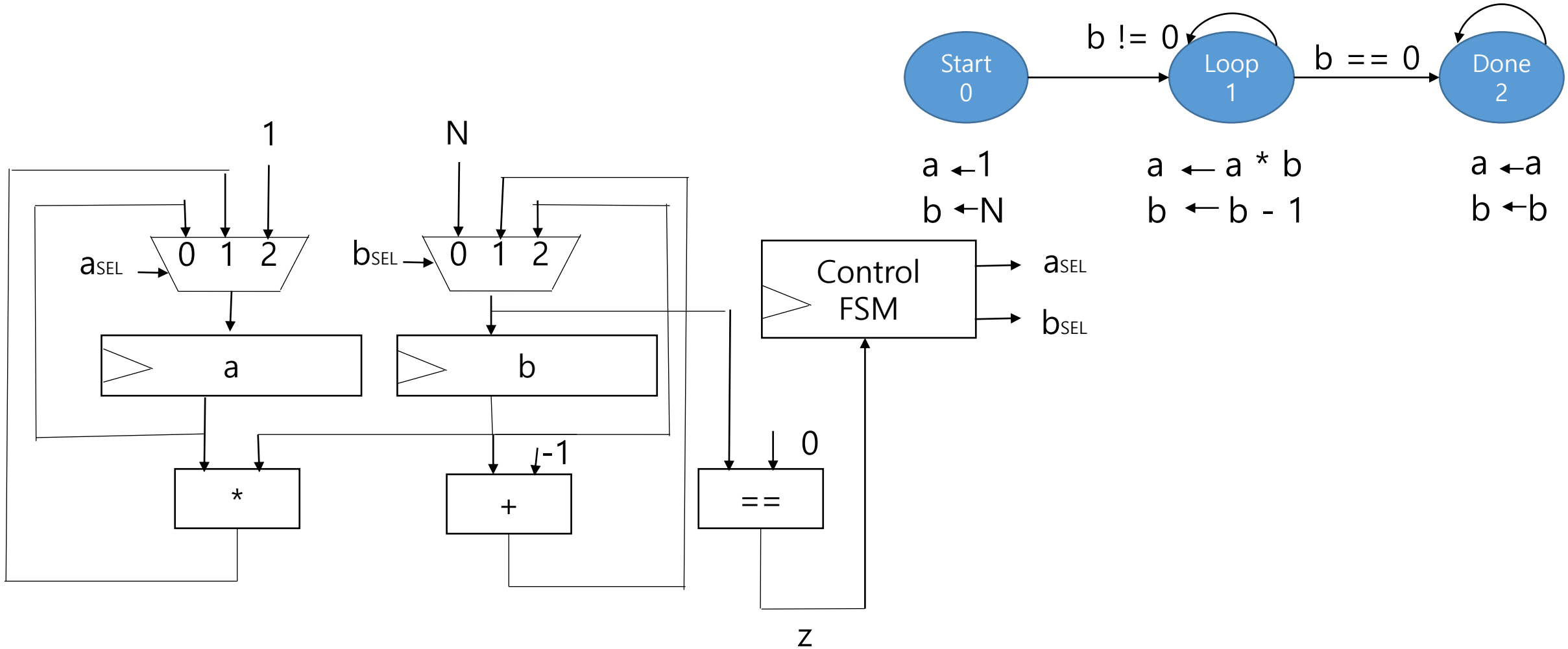
Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$



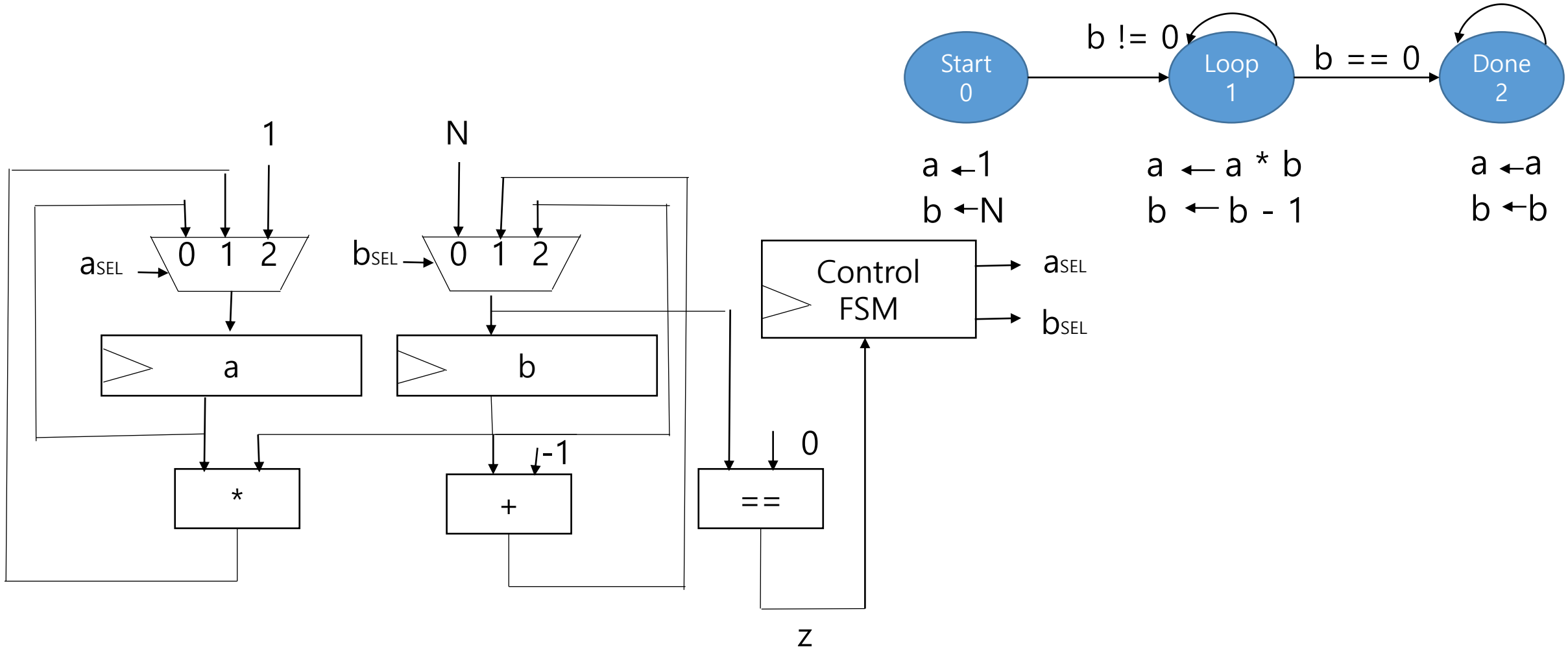
Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$



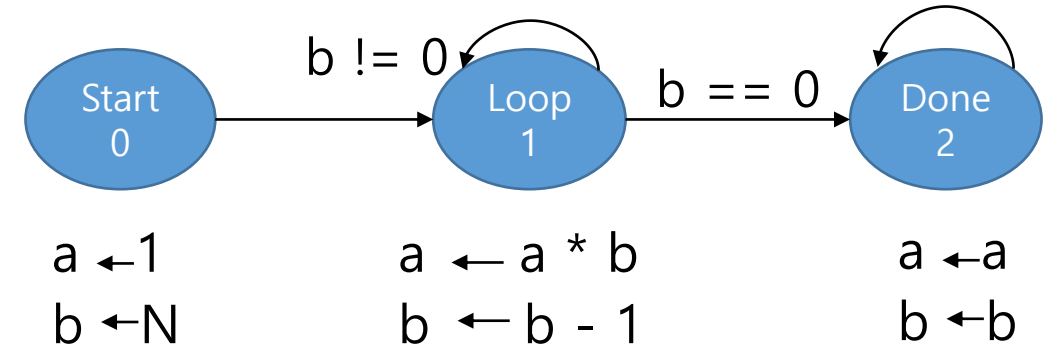
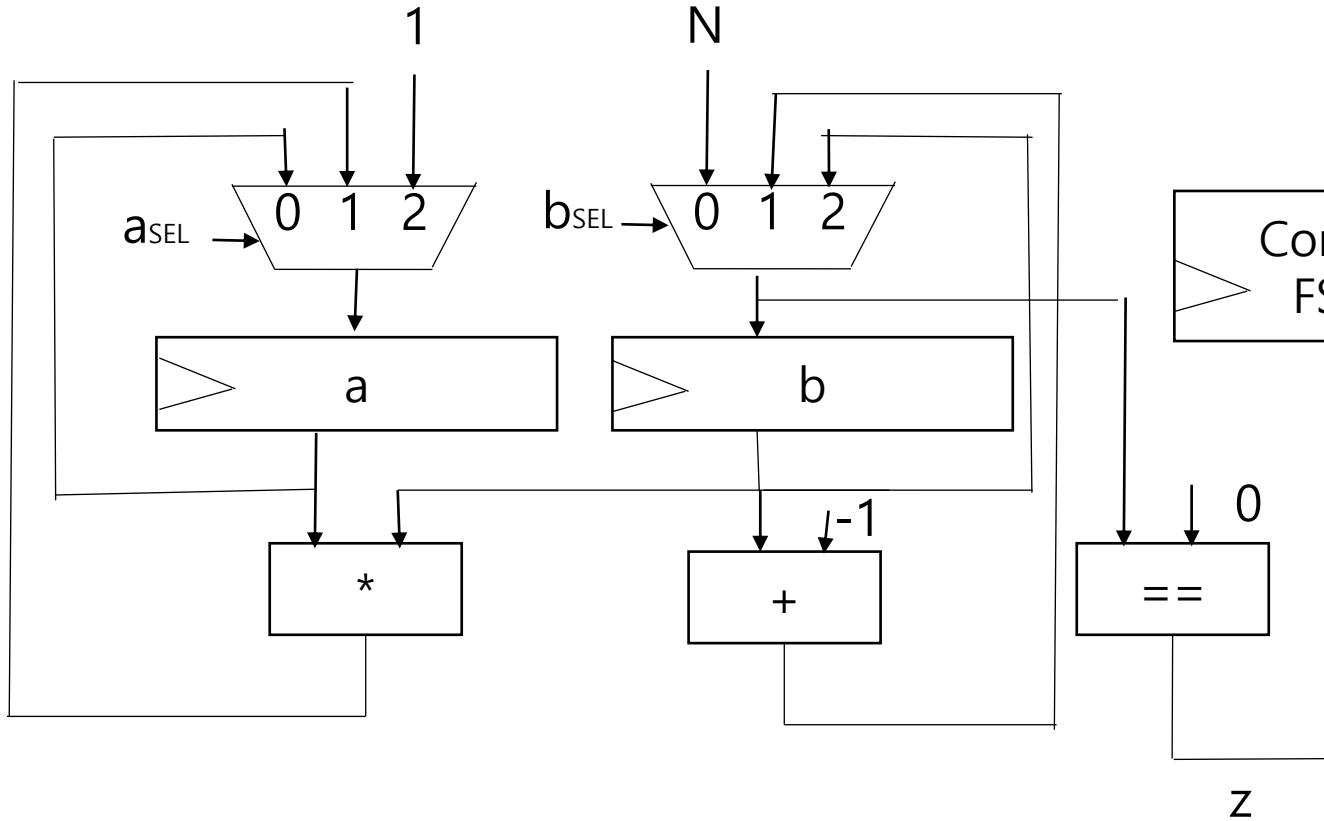
Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$



Factorial

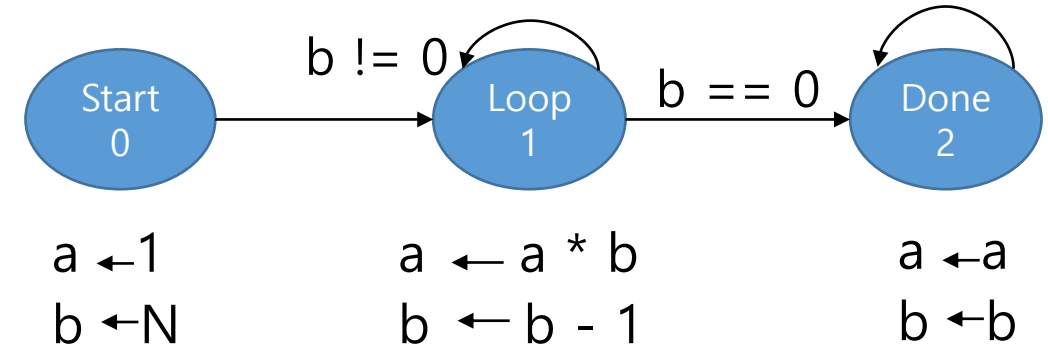
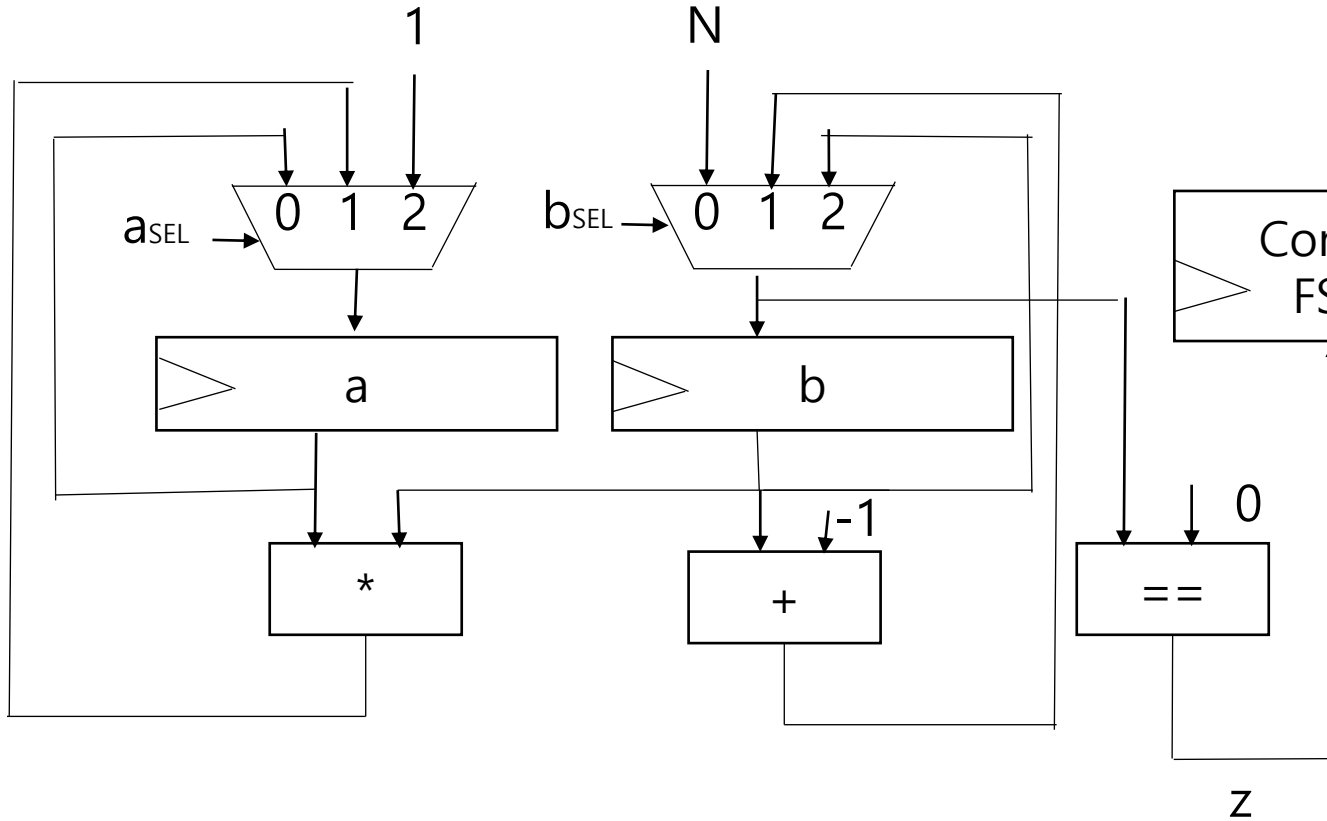
$$N! = N * (N-1) * (N-2) * \dots * 1$$



s	z	a _{SEL}	b _{SEL}	s'
00	0			
00	1			

Factorial

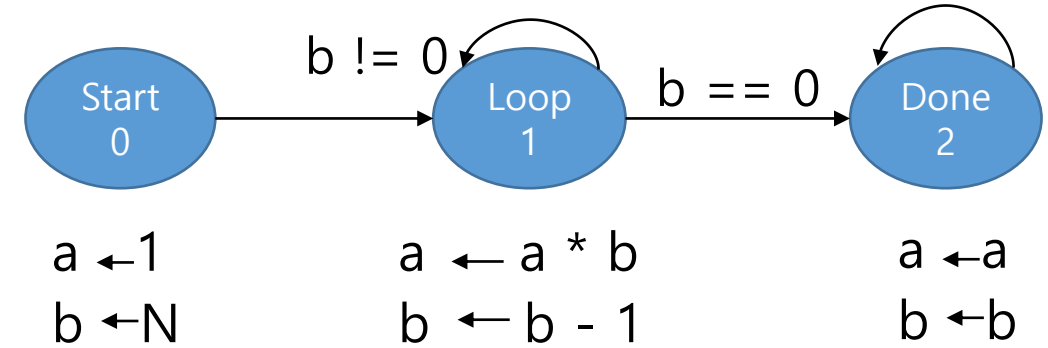
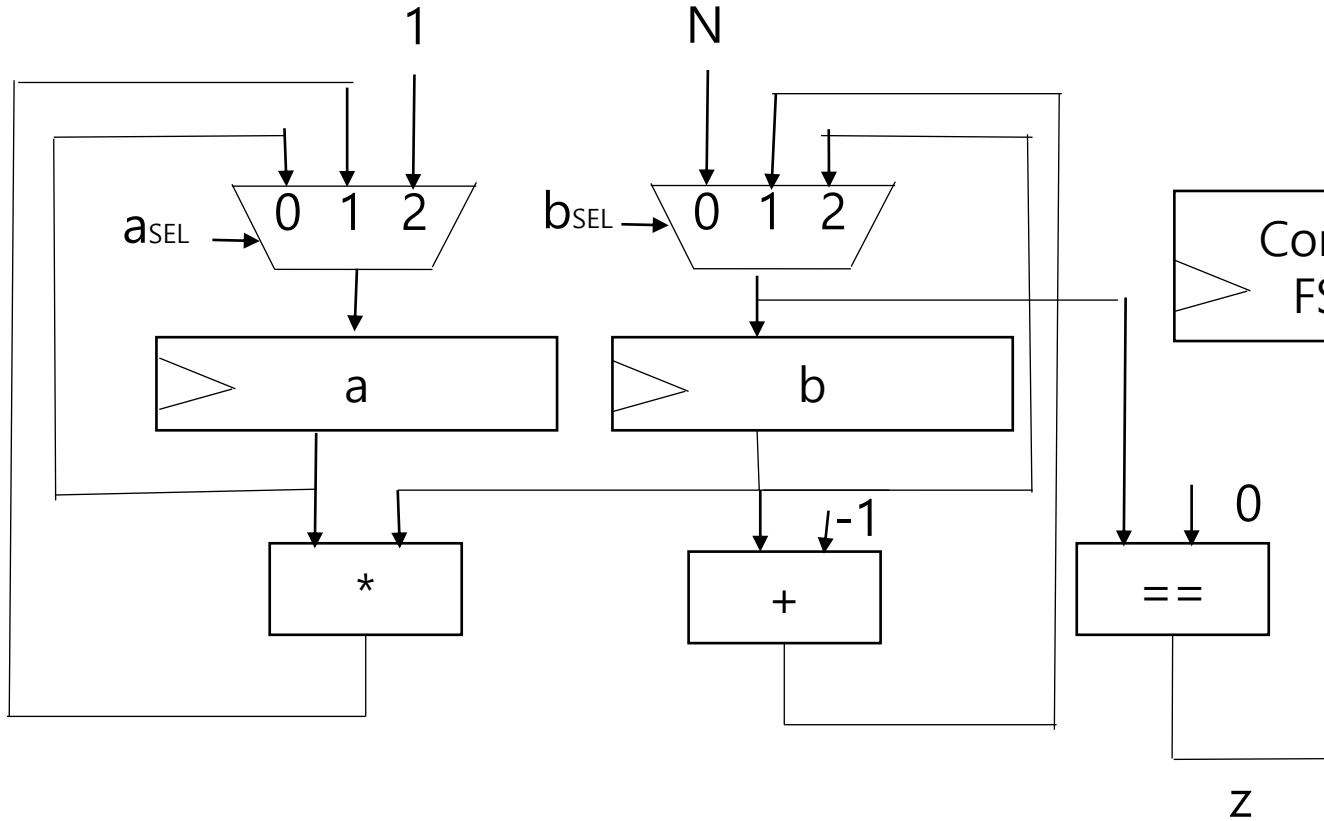
$$N! = N * (N-1) * (N-2) * \dots * 1$$



s	z	a _{SEL}	b _{SEL}	s'
00	0	10	00	01
00	1	10	00	01

Factorial

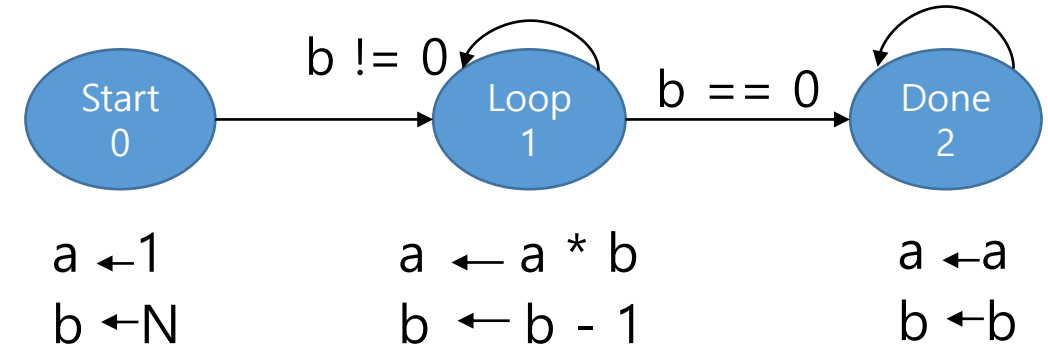
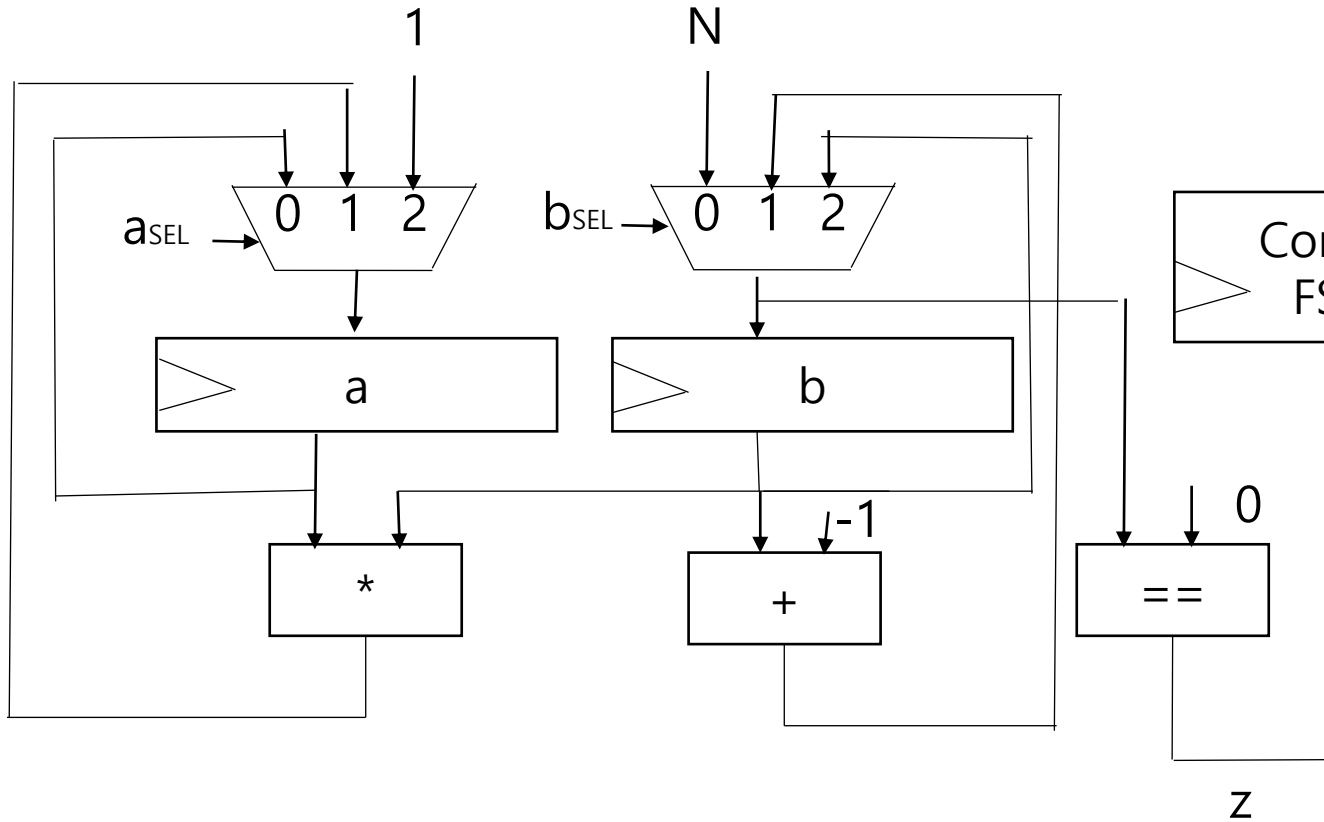
$$N! = N * (N-1) * (N-2) * \dots * 1$$



s	z	a _{SEL}	b _{SEL}	s'
01	0			
01	1			

Factorial

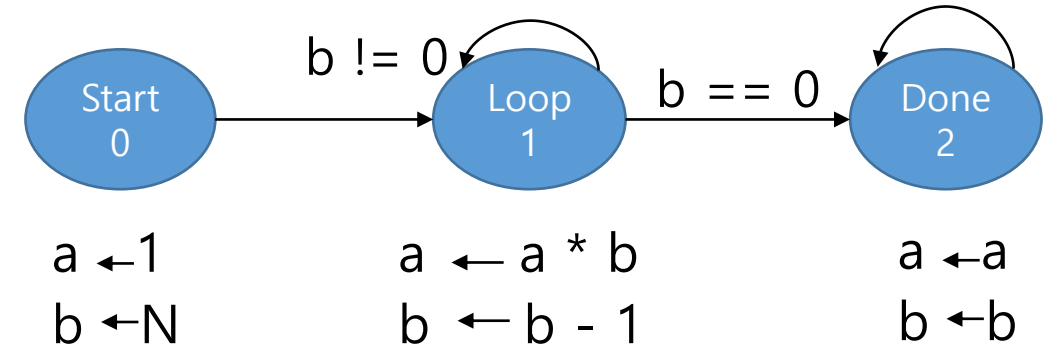
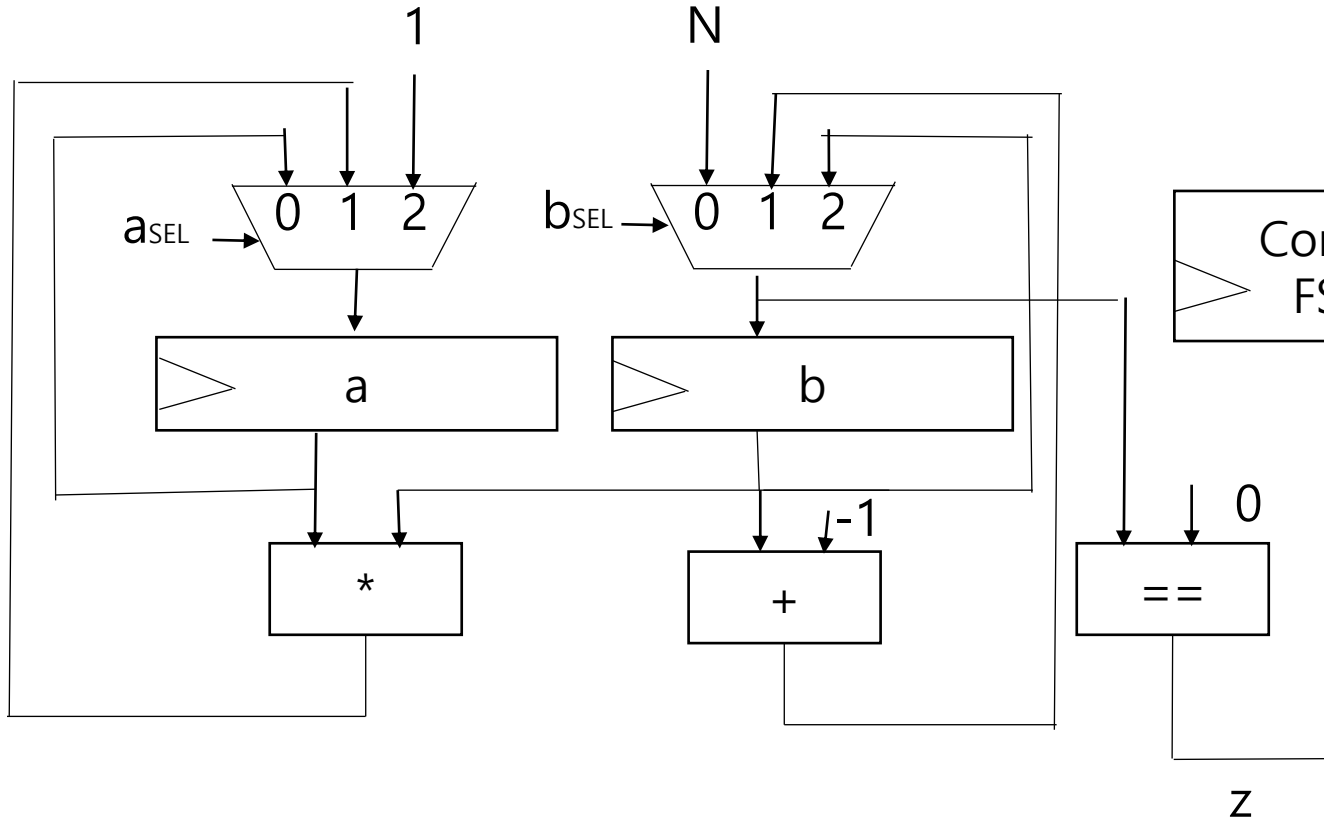
$$N! = N * (N-1) * (N-2) * \dots * 1$$



s	z	a _{SEL}	b _{SEL}	s'
01	0	01	01	01
01	1	01	01	10

Factorial

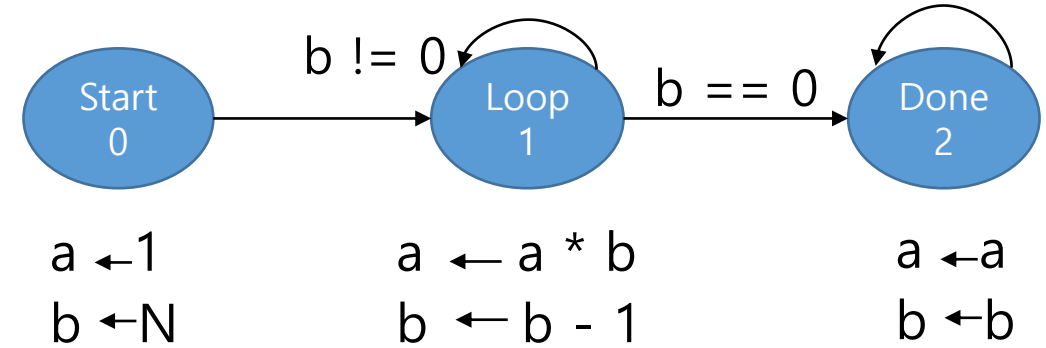
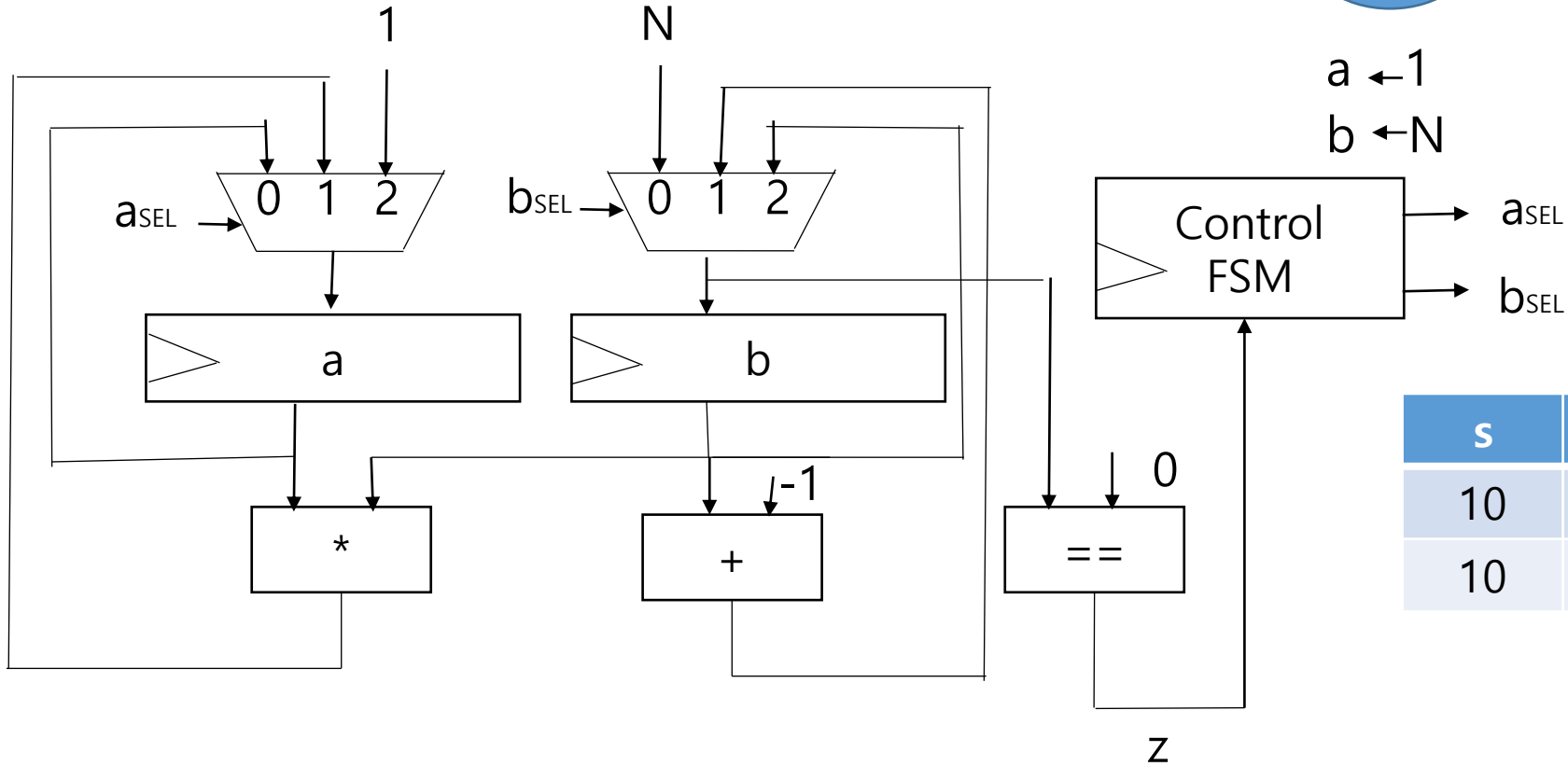
$$N! = N * (N-1) * (N-2) * \dots * 1$$



s	z	a _{SEL}	b _{SEL}	s'
10	0			
10	1			

Factorial

$$N! = N * (N-1) * (N-2) * \dots * 1$$



s	z	a_{SEL}	b_{SEL}	s'
10	0	00	10	10
10	1	00	10	10